

VYSOKÁ ŠKOLA EKONOMICKÁ V PRAZE
FAKULTA INFORMATIKY A STATISTIKY



STATISTIKA PRO INFORMATIKY

VZORCE PRO 4ST204

Obsah

1	Popisná Statistika	4
1.1	Četnosti	4
1.1.1	Absolutní četnost	4
1.1.2	Součet absolutních četností	4
1.1.3	Relativní Četnost	4
1.1.4	Součet relativních četností	4
1.1.5	Kumulativní absolutní četnost	5
1.1.6	Kumulativní relativní četnost	5
1.2	Charakteristiky polohy	6
1.2.1	Aritmetický průměr	6
1.2.2	Vážený aritmetický průměr (z četnostní tabulky)	6
1.2.3	Aritmetický průměr pomocí relativních četností	7
1.2.4	Harmonický průměr	8
1.2.5	Geometrický průměr	8
1.2.6	Vztah mezi průměry	9
1.2.7	Medián	9
1.2.8	Kvantily a kvartily	10
1.2.9	Interkvartilové rozpětí (IQR)	11
1.2.10	Modus	12
1.3	Charakteristiky variability	12
1.3.1	Variační rozpětí	12
1.3.2	Rozptyl statistického souboru	13
1.3.3	Výběrový rozptyl	14
1.3.4	Vážený rozptyl	14
1.3.5	Směrodatná odchylka statistického souboru	15
1.3.6	Výběrová směrodatná odchylka	16
1.3.7	Variační koeficient	16
1.3.8	Střední absolutní odchylka (MAD)	17
1.3.9	Detekce odlehlých hodnot – Tukeyovy meze	18
2	Teorie pravděpodobnosti	19
2.1	Počet pravděpodobnosti	19
2.1.1	Klasická definice pravděpodobnosti	19
2.1.2	Pravděpodobnost komplementárního jevu	19
2.1.3	Horní odhad pravděpodobnosti průniku	19
2.1.4	Průnik nezávislých jevů	19
2.1.5	Sjednocení dvou jevů	20
2.1.6	Sjednocení neslučitelných jevů	20
2.1.7	Podmíněná pravděpodobnost	20
2.1.8	Věta o úplné pravděpodobnosti	20
2.2	Náhodné veličiny	21
2.2.1	Pravděpodobnostní funkce diskrétní náhodné veličiny	21
2.2.2	Distribuční funkce diskrétní náhodné veličiny	21
2.2.3	Pravděpodobnost intervalu	21
2.2.4	Střední hodnota diskrétní náhodné veličiny	22
2.2.5	Rozptyl diskrétní náhodné veličiny	22
2.2.6	Hustota pravděpodobnosti spojitě náhodné veličiny	22
2.2.7	Normalizační podmínka hustoty pravděpodobnosti	23
2.2.8	Distribuční funkce spojitě náhodné veličiny	23
2.2.9	Pravděpodobnost intervalu spojitě náhodné veličiny	23

2.2.10	Střední hodnota spojité náhodné veličiny	24
2.2.11	Rozptyl spojité náhodné veličiny	24
2.2.12	Směrodatná odchylka náhodné veličiny	24
2.3	Vlastnosti střední hodnoty a rozptylu	25
2.3.1	Vlastnosti střední hodnoty $E(X)$	25
2.3.2	Vlastnosti rozptylu $D(X)$	25
3	Kombinatorika a Metriky	26
3.1	Kombinatorika	26
3.2	L^p Normy (Vzdálenosti a penalizace)	26
4	Pravděpodobnostní rozdělení	27
4.1	Diskrétní rozdělení	27
4.1.1	Binomické rozdělení $\text{Bin}(n, p)$	27
4.1.2	Poissonovo rozdělení $\text{Pois}(\lambda)$	27
4.1.3	Hypergeometrické rozdělení $\text{Hyper}(N, K, n)$	28
4.2	Spojité rozdělení	29
4.2.1	Normální rozdělení $N(\mu, \sigma^2)$	29
4.2.2	Rovnoměrné rozdělení $U(a, b)$	30
4.2.3	Exponenciální rozdělení $\text{Exp}(\lambda)$	31
4.3	Odvozená rozdělení	32
4.3.1	Studentovo t-rozdělení $t(\nu)$	32
4.3.2	Chí-kvadrát rozdělení $\chi^2(k)$	33
4.3.3	F-rozdělení $F(k_1, k_2)$	33
5	Testování hypotéz	35
5.1	Chyby testování hypotéz	35
5.2	Bodový odhad	35
5.3	Intervaly spolehlivosti a velikost výběru	35
5.3.1	Interval pro průměr (μ)	35
5.3.2	Interval pro podíl (π)	36
5.3.3	Potřebná velikost výběru (n)	37
5.4	Jednovýběrový t-test (střední hodnota)	38
5.5	Dvouvýběrový t-test (Welchův)	39
5.6	Párový t-test	39
5.7	Test proporce (Z-test)	40
5.8	Jednocestná analýza rozptylu (ANOVA)	41
5.9	Chí-kvadrát test dobré shody	42
5.10	Chí-kvadrát test nezávislosti	43
5.11	Stupně volnosti – přehled	43
6	Analýza závislosti	44
6.1	Korelační analýza	44
6.1.1	Pearsonův korelační koeficient	44
6.1.2	Koeficient determinace R^2	45
6.1.3	Spearmanův korelační koeficient	46
6.2	Jednoduchá lineární regrese	46
6.2.1	Odhad koeficientů (MNČ)	46
6.2.2	Součty čtverců a koeficient determinace R^2	47
6.2.3	t-test a F-test platnosti modelu	48
6.2.4	Predikce	48

7	Časové řady	50
7.1	Základní charakteristiky	50
7.1.1	Absolutní přírůstky a tempa růstu	50
7.1.2	Průměr časové řady	50
7.2	Řetězové a bazické indexy	51
7.3	Klouzavé průměry	51
7.3.1	Prostý klouzavý průměr (liché m)	51
7.3.2	Centrovaný klouzavý průměr (sudé m)	52
7.4	Trend a sezónnost – regresní přístup	52
7.4.1	Dummy proměnné a model	52
7.4.2	Dekompozice časové řady	53
7.4.3	Predikce na budoucí období	54
7.5	Nelineární trendy časových řad	54
7.5.1	Kvadratický (parabolický) trend	54
7.5.2	Exponenciální trend	55
8	Indexní analýza	57
8.1	Složené individuální indexy	57
8.2	Souhrnné (Laspeyresovy a Paascheho) indexy	58
8.3	Fisherův (ideální) index	59
9	Příloha: Utility funkce v Pythonu	60
9.1	Užitečné knihovny a základy	60
9.2	Práce s poli (Standardní Python)	60
9.3	Základní Python (Built-in & Math)	61
9.4	Statistika a Matice (Knihovna NumPy a jiné)	62
9.5	Pandas: Analýza dat	64
9.6	Matplotlib a Seaborn: Vizualizace	66
10	Příloha: Utility funkce v R	67
10.1	Užitečné balíčky	67
10.2	Vektory a datové typy	67
10.3	Práce s daty v R	68

1 Popisná Statistika

1.1 Četnosti

1.1.1 Absolutní četnost

Absolutní četnost i -té hodnoty je počet výskytů dané hodnoty ve výběru.

$$n_i$$

kde

- n_i je počet výskytů i -té hodnoty ve výběru.

1.1.2 Součet absolutních četností

Součet absolutních četností všech hodnot (nebo tříd) ve výběru se rovná celkovému počtu pozorování.

$$\sum_{i=1}^k n_i = n$$

kde

- $\sum$ je sumační znak (značí sčítání),
- i je index hodnoty nebo třídy,
- k je počet různých hodnot nebo tříd,
- n_i je absolutní četnost i -té hodnoty (nebo třídy),
- n je celkový počet pozorování.

1.1.3 Relativní Četnost

Relativní četnost i -té hodnoty ve výběru je definována jako podíl absolutní četnosti dané hodnoty a celkového počtu pozorování,

$$p_i = \frac{n_i}{n}$$

kde

- p_i je relativní četnost i -té hodnoty,
- n_i je absolutní četnost i -té hodnoty (kolikrát se hodnota vyskytla),
- n je celkový počet pozorování v souboru.

1.1.4 Součet relativních četností

Součet relativních četností všech hodnot (nebo tříd) ve výběru je roven jedné.

$$\sum_{i=1}^k p_i = 1$$

kde

- $\sum$ je sumační znak (značí sčítání),
- i je index hodnoty nebo třídy,
- k je počet různých hodnot nebo tříd,
- p_i je relativní četnost i -té hodnoty (nebo třídy).

1.1.5 Kumulativní absolutní četnost

Kumulativní absolutní četnost i -té hodnoty je součet absolutních četností všech hodnot až do i -té (včetně).

$$N_i = \sum_{j=1}^i n_j$$

kde

- N_i je kumulativní absolutní četnost i -té hodnoty,
- n_j je absolutní četnost j -té hodnoty,
- i je index hodnoty nebo třídy.

Implementace v R

Funkce	Příklad použití	Vysvětlení
<code>cumsum(ni)</code>	<pre>ni <- c(5, 10, 8, 7) Ni <- cumsum(ni)</pre>	Vrací kumulativní součty vektoru absolutních četností.

Implementace v Python

Funkce	Příklad použití	Vysvětlení
<code>np.cumsum(ni)</code>	<pre>import numpy as np ni = np.array([5, 10, 8, 7]) Ni = np.cumsum(ni)</pre>	Vrací kumulativní součty pole absolutních četností.

1.1.6 Kumulativní relativní četnost

Kumulativní relativní četnost i -té hodnoty je součet relativních četností všech hodnot až do i -té (včetně). Odpovídá hodnotám empirické distribuční funkce; platí $P_k = 1$.

$$P_i = \sum_{j=1}^i p_j$$

kde

- P_i je kumulativní relativní četnost i -té hodnoty,
- p_j je relativní četnost j -té hodnoty,
- i je index hodnoty nebo třídy.

Implementace v R

Funkce	Příklad použití	Vysvětlení
<code>cumsum(pi)</code>	<pre>ni <- c(5, 10, 8, 7) pi <- ni / sum(ni) Pi <- cumsum(pi)</pre>	Vrací kumulativní relativní četnosti.

Implementace v Python

Funkce	Příklad použití	Vysvětlení
<code>np.cumsum(pi)</code>	<pre>import numpy as np ni = np.array([5, 10, 8, 7]) pi = ni / ni.sum() Pi = np.cumsum(pi)</pre>	Vrací kumulativní relativní četnosti.

1.2 Charakteristiky polohy

1.2.1 Aritmetický průměr

Aritmetický průměr (výběrový průměr) je definován jako součet všech hodnot statistického souboru dělený počtem pozorování.

$$\bar{x} = \frac{\sum_{i=1}^n x_i}{n}$$

kde

- $\bar{x}$ je aritmetický průměr,
- $\sum$ je sumační znak (součet hodnot),
- x_i je i -tá hodnota statistického souboru,
- i je index hodnoty,
- n je počet pozorování.

Implementace v R

Funkce	Příklad použití	Vysvětlení
<code>mean(x)</code>	<pre>x <- c(1,2,3,4,5) mean(x)</pre>	Vrací aritmetický průměr hodnot ve vektoru x .

Implementace v Python

Funkce	Příklad použití	Vysvětlení
<code>np.mean(x)</code>	<pre>import numpy as np x = np.array([1,2,3,4,5]) np.mean(x)</pre>	Vrací aritmetický průměr pole pomocí knihovny numpy .

1.2.2 Vážený aritmetický průměr (z četnostní tabulky)

Pokud jsou data zadána pomocí četnostní tabulky, aritmetický průměr lze vypočítat jako vážený průměr hodnot.

$$\bar{x} = \frac{\sum_{i=1}^k x_i n_i}{\sum_{i=1}^k n_i}$$

kde

- $\bar{x}$ je aritmetický průměr,

- $\sum$ je sumační znak (součet),
- x_i je i -tá hodnota znaku,
- n_i je absolutní četnost i -té hodnoty,
- k je počet různých hodnot nebo tříd,
- $\sum_{i=1}^k n_i$ je celkový počet pozorování.

Implementace v R

Funkce	Příklad použití	Vysvětlení
<code>weighted.mean(x, w)</code>	<pre>x <- c(1,2,3) w <- c(2,3,5) weighted.mean(x, w)</pre>	Vrací vážený průměr hodnot ve vektoru x s váhami w .

Implementace v Python

Funkce	Příklad použití	Vysvětlení
<code>np.average(x, weights=w)</code>	<pre>import numpy as np x = np.array([1,2,3]) w = np.array([2,3,5]) np.average(x, weights=w)</pre>	Vrací vážený průměr hodnot ve vektoru x s váhami w .

1.2.3 Aritmetický průměr pomocí relativních četností

Pokud jsou data vyjádřena pomocí relativních četností, lze aritmetický průměr vypočítat jako vážený součet hodnot.

$$\bar{x} = \sum_{i=1}^k x_i p_i$$

kde

- $\bar{x}$ je aritmetický průměr,
- $\sum$ je sumační znak (součet),
- x_i je i -tá hodnota znaku,
- p_i je relativní četnost i -té hodnoty,
- i je index hodnoty,
- k je počet různých hodnot nebo tříd.

Implementace v R

Funkce	Příklad použití	Vysvětlení
<code>sum(x * p)</code>	<pre>x <- c(1,2,3) p <- c(0.2,0.3,0.5) sum(x * p)</pre>	Vrací aritmetický průměr vypočtený pomocí relativních četností.

Implementace v Python

Funkce	Příklad použití	Vysvětlení
<code>np.sum(x * p)</code>	<pre>import numpy as np x = np.array([1,2,3]) p = np.array([0.2,0.3,0.5]) np.sum(x * p)</pre>	Vrací aritmetický průměr vypočtený pomocí relativních četností pomocí knihovny <code>numpy</code> .

1.2.4 Harmonický průměr

Harmonický průměr je definován jako podíl počtu pozorování a součtu převrácených hodnot jednotlivých pozorování.

$$\bar{x}_H = \frac{n}{\sum_{i=1}^n \frac{1}{x_i}}$$

kde

- $\bar{x}_H$ je harmonický průměr,
- n je počet pozorování,
- $\sum$ je sumační znak (součet),
- x_i je i -tá hodnota statistického souboru,
- $\frac{1}{x_i}$ je převrácená hodnota i -tého pozorování.

Implementace v R

Funkce	Příklad použití	Vysvětlení
<code>n / sum(1/x)</code>	<pre>x <- c(1,2,4,5) n <- length(x) n / sum(1/x)</pre>	Vrací harmonický průměr hodnot ve vektoru <code>x</code> .

Implementace v Python

Funkce	Příklad použití	Vysvětlení
<code>scipy.stats.hmean(x)</code>	<pre>from scipy.stats import hmean x = [1,2,4,5] hmean(x)</pre>	Vrací harmonický průměr pomocí funkce <code>hmean</code> z knihovny <code>scipy.stats</code> .

1.2.5 Geometrický průměr

Geometrický průměr je definován jako n -tá odmocnina součinu všech hodnot statistického souboru.

$$\bar{x}_G = \sqrt[n]{\prod_{i=1}^n x_i} = \sqrt[n]{x_1 \cdot x_2 \cdots x_n}$$

kde

- $\bar{x}_G$ je geometrický průměr,
- $\prod$ je znak pro součin (násobení všech hodnot),
- x_i je i -tá hodnota statistického souboru,
- i je index hodnoty,
- n je počet pozorování,
- $\sqrt[n]{}$ značí n -tou odmocninu.

Implementace v R

Funkce	Příklad použití	Vysvětlení
<code>exp(mean(log(x)))</code>	<pre>x <- c(1,2,3,4) exp(mean(log(x)))</pre>	Vrací geometrický průměr hodnot ve vektoru <code>x</code> .

Implementace v Python

Funkce	Příklad použití	Vysvětlení
<code>scipy.stats.gmean(x)</code>	<pre>from scipy.stats import gmean x = [1,2,3,4] gmean(x)</pre>	Vrací geometrický průměr pomocí funkce <code>gmean</code> z knihovny <code>scipy.stats</code> .

1.2.6 Vztah mezi průměry

$$\bar{x}_H \leq \bar{x}_G \leq \bar{x}$$

kde

- $\bar{x}_H$ je harmonický průměr,
- $\bar{x}_G$ je geometrický průměr,
- $\bar{x}$ je aritmetický průměr.

1.2.7 Medián

Medián je hodnota, která rozděluje uspořádaný statistický soubor na dvě stejně početné části. Medián také vyjadřuje 50% kvantil.

$$\tilde{x} = \begin{cases} x_{(\frac{n+1}{2})}, & \text{je-li } n \text{ liché,} \\ \frac{x_{(\frac{n}{2})} + x_{(\frac{n}{2}+1)}}{2}, & \text{je-li } n \text{ sudé.} \end{cases}$$

kde

- $\tilde{x}$ je medián,
- $x_{(i)}$ je i -tá hodnota v uspořádaném statistickém souboru,
- n je počet pozorování.

Implementace v R

Funkce	Příklad použití	Vysvětlení
<code>median(x)</code>	<pre>x <- c(1,3,2,5,4) median(x)</pre>	Vrací medián hodnot ve vektoru x .

Implementace v Python

Funkce	Příklad použití	Vysvětlení
<code>np.median(x)</code>	<pre>import numpy as np x = np.array([1,3,2,5,4]) np.median(x)</pre>	Vrací medián hodnot ve vektoru x .

1.2.8 Kvantily a kvartily

Kvantily rozdělují uspořádaný statistický soubor na části se stejným počtem pozorování. q -kvantil je hodnota x_q , pro kterou platí

$$P(X \leq x_q) = q$$

kde

- x_q je q -kvantil,
- $q \in (0, 1)$ je pořadí kvantilu,
- $P(X \leq x_q)$ je pravděpodobnost, že náhodná veličina nabývá hodnot menších nebo rovných x_q .

Speciálním případem kvantilů jsou kvartily, které rozdělují data na čtyři stejně velké části.

$$Q_1 = x_{0.25}, \quad Q_2 = x_{0.5}, \quad Q_3 = x_{0.75}$$

kde

- Q_1 je první kvartil (25% dat je menších),
- Q_2 je druhý kvartil, který odpovídá mediánu,
- Q_3 je třetí kvartil (75% dat je menších).

Implementace v R

Funkce	Příklad použití	Vysvětlení
<code>quantile(x)</code>	<pre>x <- c(1,2,3,4,5,6,7,8) quantile(x)</pre>	Vrací kvantily datového vektoru.
<code>quantile(x, probs=c(0.25, 0.5, 0.75))</code>	<pre>x <- c(1,2,3,4,5,6,7,8) quantile(x, probs=c(0.25,0.5,0.75))</pre>	Vrací první, druhý a třetí kvartil.

Implementace v Python

Funkce	Příklad použití	Vysvětlení
<code>np.quantile(x,q)</code>	<pre>import numpy as np x = np.array([1,2,3,4,5,6,7,8]) np.quantile(x,0.5)</pre>	Vrací q -kvantil pole.
<code>np.quantile(x, [0.25,0.5,0.75])</code>	<pre>import numpy as np x = np.array([1,2,3,4,5,6,7,8]) np.quantile(x,[0.25,0.5,0.75])</pre>	Vrací první, druhý a třetí kvartil.

1.2.9 Interkvartilové rozpětí (IQR)

Interkvartilové rozpětí (IQR) je rozdíl mezi třetím a prvním kvantilem. Vyjadřuje rozptyl prostředních 50 % dat.

$$IQR = Q_3 - Q_1$$

kde

- IQR je interkvartilové rozpětí,
- Q_1 je první kvartil,
- Q_3 je třetí kvartil.

Implementace v R

Funkce	Příklad použití	Vysvětlení
<code>IQR(x)</code>	<pre>x <- c(1,2,3,4,5,6,7,8) IQR(x)</pre>	Vrací interkvartilové rozpětí datového vektoru.

Implementace v Python

Funkce	Příklad použití	Vysvětlení
<code>np.quantile(x,0.75)</code> - <code>np.quantile(x,0.25)</code>	<pre>import numpy as np x = np.array([1,2,3,4,5,6,7,8]) np.quantile(x,0.75) - np. quantile(x,0.25)</pre>	Vrací interkvartilové rozpětí pole.

1.2.10 Modus

Modus je hodnota, která se ve statistickém souboru vyskytuje nejčastěji (s nejvyšší absolutní četností). Soubor může mít i více modů. U spojitých proměnných se modus stanovuje po diskretizaci do intervalů.

$$\hat{x} = x_{\arg \max_i n_i}$$

kde

- $\hat{x}$ je modus,
- n_i je absolutní četnost i -té hodnoty,
- $\arg \max_i$ označuje index hodnoty s nejvyšší četností.

Implementace v R

Funkce	Příklad použití	Vysvětlení
<code>names(</code> <code>which.max(</code> <code>table(x))</code>	<pre>x <- c(1, 2, 2, 3, 3, 3, 4) names(which.max(table(x)))</pre>	Vrací modus (nejčastější hodnotu) vektoru x .

Implementace v Python

Funkce	Příklad použití	Vysvětlení
<code>statistics.mode(x)</code>	<pre>import statistics x = [1, 2, 2, 3, 3, 3, 4] statistics.mode(x)</pre>	Vrací modus pomocí standardní knihovny Pythonu.

1.3 Charakteristiky variability

1.3.1 Variační rozpětí

Variační rozpětí je rozdíl mezi maximální a minimální hodnotou statistického souboru.

$$R = x_{\max} - x_{\min}$$

kde

- R je variační rozpětí,
- $x_{\max}$ je maximální hodnota v souboru,
- $x_{\min}$ je minimální hodnota v souboru.

Implementace v R

Funkce	Příklad použití	Vysvětlení
<code>max(x) - min(x)</code>	<pre>x <- c(3,7,2,9,5) max(x) - min(x)</pre>	Vrací variační rozpětí hodnot ve vektoru <code>x</code> .

Implementace v Python

Funkce	Příklad použití	Vysvětlení
<code>np.max(x) - np.min(x)</code>	<pre>import numpy as np x = np.array([3,7,2,9,5]) np.max(x) - np.min(x)</pre>	Vrací variační rozpětí hodnot ve vektoru <code>x</code> .

1.3.2 Rozptyl statistického souboru

Rozptyl vyjadřuje průměrnou kvadratickou odchylku hodnot od aritmetického průměru.

$$s_x^2 = \frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n}$$

kde

- s_x^2 je rozptyl statistického souboru,
- $\sum$ je sumační znak (součet),
- x_i je i -tá hodnota statistického souboru,
- $\bar{x}$ je aritmetický průměr,
- $x_i - \bar{x}$ je odchylka hodnoty od průměru,
- $(x_i - \bar{x})^2$ je čtverec odchylky,
- n je počet pozorování.

Výpočetní tvar rozptylu:

$$s_x^2 = \frac{\sum_{i=1}^n x_i^2}{n} - \left(\frac{\sum_{i=1}^n x_i}{n} \right)^2$$

Implementace v R

Funkce	Příklad použití	Vysvětlení
<code>mean((x - mean(x))^2)</code>	<pre>x <- c(2,4,6,8) mean((x-mean(x))^2)</pre>	Vrací nevýběrový (populační) rozptyl hodnot ve vektoru <code>x</code> .

Implementace v Python

Funkce	Příklad použití	Vysvětlení
<code>np.var(x)</code>	<pre>import numpy as np x = np.array([2,4,6,8]) np.var(x)</pre>	Vrací nevýběrový (populační) rozptyl pomocí knihovny <code>numpy</code> .

1.3.3 Výběrový rozptyl

Výběrový rozptyl je nezkreslený odhad rozptylu populace a vznikne úpravou rozptylu statistického souboru pomocí Besselovy korekce.

$$S^2 = \frac{n}{n-1} s_x^2$$

kde

- S^2 je výběrový rozptyl,
- s_x^2 je rozptyl statistického souboru,
- n je počet pozorování,
- $n-1$ je korekce při odhadu rozptylu (Besselova korekce).

Implementace v R

Funkce	Příklad použití	Vysvětlení
<code>var(x)</code>	<pre>x <- c(2,4,6,8) var(x)</pre>	Vrací výběrový rozptyl hodnot ve vektoru <code>x</code> (dělí se $n-1$).

Implementace v Python

Funkce	Příklad použití	Vysvětlení
<code>np.var(x, ddof=1)</code>	<pre>import numpy as np x = np.array([2,4,6,8]) np.var(x, ddof=1)</pre>	Vrací výběrový rozptyl pomocí knihovny <code>numpy</code> (Besselova korekce).

1.3.4 Vážený rozptyl

Vážený rozptyl vyjadřuje variabilitu hodnot znaku, když mají jednotlivé hodnoty různé váhy nebo relativní četnosti.

$$s^2 = \sum_{i=1}^k p_i (x_i - \bar{x})^2$$

kde

- s^2 je vážený rozptyl,

- x_i je i -tá hodnota znaku,
- $\bar{x}$ je vážený aritmetický průměr,
- p_i je váha nebo relativní četnost i -té hodnoty,
- k je počet různých hodnot nebo tříd.

Implementace v R

Funkce	Příklad použití	Vysvětlení
<code>weighted.mean((x-mean(x))^2, w)</code>	<pre>x <- c(1,2,3,4) w <- c(0.1,0.2,0.3,0.4) weighted.mean((x-weighted.mean(x, w))^2, w)</pre>	Vrací vážený rozptyl hodnot s váhami w .

Implementace v Python

Funkce	Příklad použití	Vysvětlení
<code>np.average((x-xbar)**2, weights=w)</code>	<pre>import numpy as np x = np.array([1,2,3,4]) w = np.array([0.1,0.2,0.3,0.4]) xbar = np.average(x, weights=w) np.average((x-xbar)**2, weights=w)</pre>	Vrací vážený rozptyl hodnot s váhami w pomocí knihovny numpy.

1.3.5 Směrodatná odchylka statistického souboru

Směrodatná odchylka je druhá odmocnina rozptylu statistického souboru.

$$s_x = \sqrt{s_x^2}$$

kde

- s_x je směrodatná odchylka,
- s_x^2 je rozptyl statistického souboru,
- $\sqrt{\cdot}$ značí druhou odmocninu.

Implementace v R

Funkce	Příklad použití	Vysvětlení
<code>sqrt(mean((x-mean(x))^2))</code>	<pre>x <- c(2,4,6,8) sqrt(mean((x-mean(x))^2))</pre>	Vrací směrodatnou odchylku statistického souboru.

Implementace v Python

Funkce	Příklad použití	Vysvětlení
<code>np.std(x)</code>	<pre>import numpy as np x = np.array([2,4,6,8]) np.std(x)</pre>	Vrací směrodatnou odchylku pole (nevýběrovou).

1.3.6 Výběrová směrodatná odchylka

Výběrová směrodatná odchylka je odmocnina z výběrového rozptylu.

$$S = \sqrt{S^2}$$

kde

- S je výběrová směrodatná odchylka,
- S^2 je výběrový rozptyl,
- $\sqrt{\cdot}$ značí druhou odmocninu.

Implementace v R

Funkce	Příklad použití	Vysvětlení
<code>sd(x)</code>	<pre>x <- c(2,4,6,8) sd(x)</pre>	Vrací výběrovou směrodatnou odchylku (dělí se $n - 1$).

Implementace v Python

Funkce	Příklad použití	Vysvětlení
<code>np.std(x, ddof=1)</code>	<pre>import numpy as np x = np.array([2,4,6,8]) np.std(x, ddof=1)</pre>	Vrací výběrovou směrodatnou odchylku (Besselova korekce).

1.3.7 Variační koeficient

Variační koeficient vyjadřuje relativní variabilitu dat a udává poměr směrodatné odchylky k aritmetickému průměru.

$$v_x = \frac{s_x}{\bar{x}}$$

kde

- v_x je variační koeficient,
- s_x je směrodatná odchylka,
- $\bar{x}$ je aritmetický průměr.

Implementace v R

Funkce	Příklad použití	Vysvětlení
<code>sd(x) / mean(x)</code>	<pre>x <- c(2,4,6,8) sd(x) / mean(x)</pre>	Vrací variační koeficient hodnot ve vektoru <code>x</code> .

Implementace v Python

Funkce	Příklad použití	Vysvětlení
<code>np.std(x, ddof=1) / np.mean(x)</code>	<pre>import numpy as np x = np.array([2,4,6,8]) np.std(x, ddof=1) / np.mean(x)</pre>	Vrací variační koeficient hodnot ve vektoru <code>x</code> . <code>ddof=1</code> odpovídá výběrové SD (dělí $n-1$), stejně jako <code>sd()</code> v R.

1.3.8 Střední absolutní odchylka (MAD)

Střední absolutní odchylka (medián absolutních odchylek od mediánu) je robustní míra variability odolná vůči odlehlým hodnotám.

$$\text{MAD} = \text{median}(|x_i - \tilde{x}|)$$

kde

- MAD je střední absolutní odchylka,
- x_i jsou hodnoty statistického souboru,
- $\tilde{x}$ je medián,
- $|\cdot|$ označuje absolutní hodnotu.

Implementace v R

Funkce	Příklad použití	Vysvětlení
<code>mad(x)</code>	<pre>x <- c(2, 4, 6, 8, 100) mad(x) # škálovaný (x 1.4826) mad(x, constant=1)# neškálovaný MAD</pre>	R automaticky škáluje MAD faktorem 1,4826 pro konzistenci se σ normálního rozdělení.

Implementace v Python

Funkce	Příklad použití	Vysvětlení
<code>np.median(np.abs(x - np.median(x)))</code>	<pre>import numpy as np x = np.array([2, 4, 6, 8, 100]) np.median(np.abs(x - np.median(x)))</pre>	Neškálovaný MAD pomocí knihovny numpy.

1.3.9 Detekce odlehlých hodnot – Tukeyovy meze

Hodnota je považována za odlehlou (outlier), pokud leží mimo interval ohraničený Tukeyovými mezemi. Tyto meze zobrazuje boxplot automaticky.

$$\text{dolní mez} = Q_1 - 1,5 \cdot IQR, \quad \text{horní mez} = Q_3 + 1,5 \cdot IQR$$

kde

- Q_1 je první kvartil,
- Q_3 je třetí kvartil,
- $IQR = Q_3 - Q_1$ je interkvartilové rozpětí.

Implementace v R

Funkce	Příklad použití	Vysvětlení
Tukeyovy meze	<pre>q1 <- quantile(x, 0.25) q3 <- quantile(x, 0.75) iqr <- IQR(x) dolni <- q1 - 1.5 * iqr horni <- q3 + 1.5 * iqr outliers <- x[x < dolni x > horni]</pre>	Identifikace odlehlých hodnot. Boxplot tyto meze zobrazuje automaticky jako vousy.

Implementace v Python

Funkce	Příklad použití	Vysvětlení
Tukeyovy meze	<pre>import numpy as np q1, q3 = np.quantile(x, [0.25, 0.75]) iqr = q3 - q1 outliers = x[(x < q1 - 1.5*iqr) (x > q3 + 1.5*iqr)]</pre>	Identifikace odlehlých hodnot.

2 Teorie pravděpodobnosti

2.1 Počet pravděpodobnosti

2.1.1 Klasická definice pravděpodobnosti

Pravděpodobnost jevu A při klasické definici pravděpodobnosti je podíl počtu příznivých výsledků a celkového počtu stejně pravděpodobných výsledků.

$$P(A) = \frac{m}{n}$$

kde

- $P(A)$ je pravděpodobnost jevu A ,
 - m je počet příznivých výsledků,
 - n je počet všech možných výsledků.
-

2.1.2 Pravděpodobnost komplementárního jevu

Pravděpodobnost opačného (komplementárního, doplňkového) jevu k jevu A je rovna jedné minus pravděpodobnost jevu A .

$$P(\overline{A}) = 1 - P(A)$$

kde

- $P(\overline{A})$ je pravděpodobnost opačného jevu k A ,
 - $P(A)$ je pravděpodobnost jevu A .
-

2.1.3 Horní odhad pravděpodobnosti průniku

Pravděpodobnost současného nastání dvou jevů nemůže být větší než pravděpodobnost kteréhokolik z nich.

$$P(A \cap B) \leq \min(P(A), P(B))$$

kde

- $P(A \cap B)$ je pravděpodobnost, že nastanou oba jevy současně,
 - $P(A)$ je pravděpodobnost jevu A ,
 - $P(B)$ je pravděpodobnost jevu B ,
 - $\min(\cdot)$ označuje minimum z daných hodnot.
-

2.1.4 Průnik nezávislých jevů

Jsou-li jevy A a B nezávislé, pak pravděpodobnost jejich současného nastání je rovna součinu jejich pravděpodobností.

$$P(A \cap B) = P(A)P(B)$$

kde

- $P(A \cap B)$ je pravděpodobnost současného nastání jevů A a B ,
 - $P(A)$ je pravděpodobnost jevu A ,
 - $P(B)$ je pravděpodobnost jevu B .
-

2.1.5 Sjednocení dvou jevů

Pravděpodobnost, že nastane alespoň jeden z jevů A nebo B , je rovna součtu jejich pravděpodobností minus pravděpodobnost jejich současného nastání.

$$P(A \cup B) = P(A) + P(B) - P(A \cap B)$$

kde

- $P(A \cup B)$ je pravděpodobnost, že nastane alespoň jeden z jevů,
 - $P(A)$ je pravděpodobnost jevu A ,
 - $P(B)$ je pravděpodobnost jevu B ,
 - $P(A \cap B)$ je pravděpodobnost současného nastání jevů A a B .
-

2.1.6 Sjednocení neslučitelných jevů

Pokud jsou jevy A a B neslučitelné (nemohou nastat současně), pak pravděpodobnost jejich sjednocení je rovna součtu pravděpodobností těchto jevů.

$$P(A \cup B) = P(A) + P(B)$$

kde

- $P(A \cup B)$ je pravděpodobnost, že nastane jev A nebo B ,
 - $P(A)$ je pravděpodobnost jevu A ,
 - $P(B)$ je pravděpodobnost jevu B .
-

2.1.7 Podmíněná pravděpodobnost

Podmíněná pravděpodobnost jevu A za podmínky, že nastal jev B , je pravděpodobnost současného nastání jevů A a B dělená pravděpodobností B .

$$P(A | B) = \frac{P(A \cap B)}{P(B)}, \quad P(B) > 0$$

kde

- $P(A | B)$ je podmíněná pravděpodobnost jevu A za podmínky B ,
- $P(A \cap B)$ je pravděpodobnost současného nastání jevů A a B ,
- $P(B)$ je pravděpodobnost jevu B .

Jevy A a B jsou **nezávislé**, právě když $P(A | B) = P(A)$, tj. nastání B nemění pravděpodobnost A .

2.1.8 Věta o úplné pravděpodobnosti

Je-li $\{B_1, B_2, \dots, B_k\}$ úplný systém jevů (vzájemně se vylučují a jejich sjednocení je jistý jev), pak pro libovolný jev A platí

$$P(A) = \sum_{j=1}^k P(A | B_j) P(B_j)$$

kde

- $P(A)$ je pravděpodobnost jevu A ,
- $B_1, \dots, B_k$ jsou vzájemně se vylučující jevy pokrývající celý prostor,
- $P(A | B_j)$ je podmíněná pravděpodobnost A za podmínky B_j ,
- $P(B_j)$ je pravděpodobnost jevu B_j .

2.2 Náhodné veličiny

2.2.1 Pravděpodobnostní funkce diskrétní náhodné veličiny

Pravděpodobnost, že diskrétní náhodná veličina X nabývá konkrétní hodnoty x .

$$P(x) = P(X = x)$$

kde

- $P(x)$ je pravděpodobnost hodnoty x ,
 - $P(X = x)$ je pravděpodobnost, že náhodná veličina X má hodnotu x ,
 - X je náhodná veličina,
 - x je konkrétní hodnota náhodné veličiny.
-

2.2.2 Distribuční funkce diskrétní náhodné veličiny

Distribuční funkce (kumulativní pravděpodobnost) udává pravděpodobnost, že náhodná veličina X bude menší nebo rovna hodnotě x .

$$F(x) = P(X \leq x) = \sum_{x_j \leq x} P(x_j)$$

kde

- $F(x)$ je distribuční funkce,
 - $P(X \leq x)$ je pravděpodobnost, že X je menší nebo rovno x ,
 - $\sum$ je součet pravděpodobností,
 - x_j jsou jednotlivé možné hodnoty náhodné veličiny,
 - $P(x_j)$ je pravděpodobnost, že $X = x_j$.
-

2.2.3 Pravděpodobnost intervalu

Pravděpodobnost, že náhodná veličina X nabývá hodnot v intervalu $(x_1, x_2]$.

$$P(x_1 < X \leq x_2) = \sum_{x_1 < x \leq x_2} P(x) = F(x_2) - F(x_1)$$

kde

- $P(x_1 < X \leq x_2)$ je pravděpodobnost, že X leží mezi x_1 a x_2 ,
 - X je náhodná veličina,
 - x_1, x_2 jsou hranice intervalu,
 - $\sum$ je součet pravděpodobností všech hodnot v intervalu,
 - $P(x)$ je pravděpodobnost, že $X = x$,
 - $F(x)$ je distribuční funkce.
-

2.2.4 Střední hodnota diskrétní náhodné veličiny

Střední hodnota (očekávaná hodnota) diskrétní náhodné veličiny je vážený průměr všech možných hodnot, kde váhami jsou jejich pravděpodobnosti.

$$E(X) = \sum_x x P(x)$$

kde

- $E(X)$ je očekávaná hodnota náhodné veličiny X ,
 - $\sum$ je součet přes všechny možné hodnoty,
 - x jsou jednotlivé možné hodnoty náhodné veličiny,
 - $P(x) = P(X = x)$ je pravděpodobnost, že X nabude hodnoty x .
-

2.2.5 Rozptyl diskrétní náhodné veličiny

Rozptyl náhodné veličiny vyjadřuje míru variability hodnot náhodné veličiny kolem její střední hodnoty.

$$D(X) = E(X^2) - [E(X)]^2 = \sum_x x^2 P(x) - \left(\sum_x x P(x) \right)^2$$

kde

- $D(X)$ je rozptyl náhodné veličiny X ,
 - $E(X)$ je očekávaná hodnota náhodné veličiny,
 - $E(X^2)$ je očekávaná hodnota druhé mocniny náhodné veličiny,
 - $\sum$ je součet přes všechny možné hodnoty,
 - x jsou jednotlivé možné hodnoty náhodné veličiny,
 - $P(x) = P(X = x)$ je pravděpodobnost, že X nabude hodnoty x .
-

2.2.6 Hustota pravděpodobnosti spojitě náhodné veličiny

Hustota pravděpodobnosti vyjadřuje rychlost změny distribuční funkce spojitě náhodné veličiny.

$$f(x) = F'(x)$$

kde

- $f(x)$ je hustota pravděpodobnosti,
 - $F(x)$ je distribuční funkce,
 - $F'(x)$ je derivace distribuční funkce podle x ,
 - x je hodnota náhodné veličiny.
-

2.2.7 Normalizační podmínka hustoty pravděpodobnosti

Celková pravděpodobnost spojité náhodné veličiny přes všechny možné hodnoty je rovna jedné.

$$\int_{-\infty}^{\infty} f(x) dx = 1$$

kde

- $\int$ označuje integrál (součet přes všechny hodnoty),
 - $f(x)$ je hustota pravděpodobnosti,
 - dx je integrační proměnná,
 - $-\infty, \infty$ vyjadřují integraci přes všechny možné hodnoty náhodné veličiny.
-

2.2.8 Distribuční funkce spojité náhodné veličiny

Distribuční funkce spojité náhodné veličiny udává pravděpodobnost, že náhodná veličina X bude menší nebo rovna hodnotě x .

$$F(x) = P(X \leq x) = \int_{-\infty}^x f(t) dt$$

kde

- $F(x)$ je distribuční funkce,
 - $P(X \leq x)$ je pravděpodobnost, že náhodná veličina X je menší nebo rovna x ,
 - $\int$ označuje integrál,
 - $f(t)$ je hustota pravděpodobnosti,
 - t je integrační proměnná,
 - $-\infty$ je dolní mez integrace,
 - x je horní mez integrace.
-

2.2.9 Pravděpodobnost intervalu spojité náhodné veličiny

Pravděpodobnost, že spojitá náhodná veličina X nabývá hodnot v intervalu $(x_1, x_2]$.

$$P(x_1 < X \leq x_2) = \int_{x_1}^{x_2} f(x) dx = F(x_2) - F(x_1)$$

kde

- $P(x_1 < X \leq x_2)$ je pravděpodobnost, že X leží mezi x_1 a x_2 ,
 - X je náhodná veličina,
 - x_1, x_2 jsou hranice intervalu,
 - $\int$ označuje integrál,
 - $f(x)$ je hustota pravděpodobnosti,
 - dx je integrační proměnná,
 - $F(x)$ je distribuční funkce.
-

2.2.10 Střední hodnota spojitě náhodné veličiny

Střední hodnota (očekávaná hodnota) spojitě náhodné veličiny je vážený průměr všech možných hodnot, kde váhami jsou hodnoty hustoty pravděpodobnosti.

$$E(X) = \int_{-\infty}^{\infty} x f(x) dx$$

kde

- $E(X)$ je očekávaná hodnota náhodné veličiny X ,
 - $\int$ označuje integrál,
 - x je hodnota náhodné veličiny,
 - $f(x)$ je hustota pravděpodobnosti,
 - dx je integrační proměnná,
 - $-\infty, \infty$ vyjadřují integraci přes všechny možné hodnoty náhodné veličiny.
-

2.2.11 Rozptyl spojitě náhodné veličiny

Rozptyl spojitě náhodné veličiny vyjadřuje variabilitu hodnot náhodné veličiny kolem její střední hodnoty.

$$D(X) = \int_{-\infty}^{\infty} x^2 f(x) dx - \left[\int_{-\infty}^{\infty} x f(x) dx \right]^2$$

kde

- $D(X)$ je rozptyl náhodné veličiny X ,
 - $\int$ označuje integrál,
 - x je hodnota náhodné veličiny,
 - $f(x)$ je hustota pravděpodobnosti,
 - dx je integrační proměnná,
 - $\int x f(x) dx$ je očekávaná hodnota $E(X)$,
 - $\int x^2 f(x) dx$ je očekávaná hodnota $E(X^2)$.
-

2.2.12 Směrodatná odchylka náhodné veličiny

Směrodatná odchylka vyjadřuje míru variability hodnot náhodné veličiny kolem její střední hodnoty.

$$\sigma = \sqrt{D(X)}$$

kde

- σ je směrodatná odchylka,
- $D(X)$ je rozptyl náhodné veličiny X ,
- $\sqrt{}$ označuje druhou odmocninu.

2.3 Vlastnosti střední hodnoty a rozptylu

2.3.1 Vlastnosti střední hodnoty $E(X)$

Pro libovolné náhodné veličiny X, Y a konstantu c platí:

$$E(c) = c, \quad E(cX) = cE(X), \quad E(X + c) = E(X) + c$$
$$E(X + Y) = E(X) + E(Y)$$

Pro **nezávislé** náhodné veličiny X a Y navíc platí:

$$E(X \cdot Y) = E(X) \cdot E(Y)$$

kde

- $E(\cdot)$ označuje střední hodnotu (očekávanou hodnotu),
- c je konstanta,
- X, Y jsou náhodné veličiny.

Implementace v R

Funkce	Příklad použití	Vysvětlení
<code>mean(c * X)</code>	<pre>c1 <- 3 X <- c(1, 2, 3, 4, 5) mean(c1 * X) # = c1 * mean(X) mean(X + c1) # = mean(X) + c1</pre>	Ověření linearity střední hodnoty.

2.3.2 Vlastnosti rozptylu $D(X)$

Pro libovolnou náhodnou veličinu X a konstantu c platí:

$$D(c) = 0, \quad D(cX) = c^2 D(X), \quad D(X + c) = D(X)$$

Pro **nezávislé** náhodné veličiny X a Y navíc platí:

$$D(X + Y) = D(X) + D(Y)$$

kde

- $D(\cdot)$ označuje rozptyl,
- c je konstanta,
- X, Y jsou nezávislé náhodné veličiny.

Implementace v R

Funkce	Příklad použití	Vysvětlení
<code>var_pop(c * X)</code>	<pre>var_pop <- function(x) mean((x- mean(x))^2) var_pop(c1 * X) # = c1^2 * var_pop(X) var_pop(X + c1) # = var_pop(X)</pre>	Vlastnosti populačního rozptylu (dělí n); <code>var()</code> v R dělí $n - 1$.

3 Kombinatorika a Metriky

3.1 Kombinatorika

Mějme množinu n prvků, ze které vybíráme k prvků.

- **Permutace (Pořadí všech n prvků):** Bez opakování: $P(n) = n!$
S opakováním: $P'_{k_1, \dots, k_m}(n) = \frac{n!}{k_1! \dots k_m!}$
- **Variace (Výběr k prvků z n , záleží na pořadí):** Bez opakování: $V(k, n) = \frac{n!}{(n-k)!}$
S opakováním: $V'(k, n) = n^k$
- **Kombinace (Výběr k prvků z n , nezáleží na pořadí):** Bez opakování: $K(k, n) = \binom{n}{k} = \frac{n!}{k!(n-k)!}$
S opakováním: $K'(k, n) = \binom{n+k-1}{k} = \frac{(n+k-1)!}{k!(n-1)!}$

3.2 L^p Normy (Vzdálenosti a penalizace)

Obecně: $\|x\|_p = (\sum |x_i|^p)^{1/p}$. Klíčové pro metriky chyb a regularizaci.

- L^1 (**Manhattan**): $\|x\|_1 = \sum |x_i|$. Využití: Lasso regrese.
- L^2 (**Euklid**): $\|x\|_2 = \sqrt{\sum x_i^2}$. Využití: Ridge regrese (metoda nejmenších čtverců).
- L^∞ (**Max**): $\|x\|_\infty = \max(|x_i|)$. Reprezentuje nejhorší možnou chybu.

4 Pravděpodobnostní rozdělení

4.1 Diskrétní rozdělení

4.1.1 Binomické rozdělení $\text{Bin}(n, p)$

Popisuje počet úspěchů v n nezávislých pokusech, kde pravděpodobnost úspěchu v každém pokusu je p .

$$P(X = k) = \binom{n}{k} p^k (1-p)^{n-k}, \quad k = 0, 1, \dots, n$$

$$E(X) = np, \quad D(X) = np(1-p)$$

kde

- n je počet pokusů,
- p je pravděpodobnost úspěchu v jednom pokusu,
- k je počet úspěchů,
- $\binom{n}{k} = \frac{n!}{k!(n-k)!}$ je kombinační číslo.

Implementace v R

Funkce	Příklad použití	Vysvětlení
<code>dbinom(k, n, p)</code>	<pre>dbinom(3, size=10, prob=0.3) # P(X=3) pbinom(3, size=10, prob=0.3) # P(X<=3) qbinom(0.9, size=10, prob=0.3) # kvantil rbinom(100, size=10, prob=0.3) # simulace</pre>	d/p/q/r = hustota / distribuční fce / kvantilová fce / simulace.

Implementace v Python

Funkce	Příklad použití	Vysvětlení
<code>stats.binom.pmf(k, n, p)</code>	<pre>from scipy.stats import binom binom.pmf(3, n=10, p=0.3) # P(X=3) binom.cdf(3, n=10, p=0.3) # P(X<=3) binom.ppf(0.9, n=10, p=0.3) # kvantil</pre>	Binomické rozdělení z knihovny <code>scipy.stats</code> .

4.1.2 Poissonovo rozdělení $\text{Pois}(\lambda)$

Popisuje počet náhodných událostí v daném časovém intervalu nebo oblasti, kdy průměrný počet událostí je λ . Vhodné pro vzácné jevy (velké n , malé p).

$$P(X = k) = \frac{\lambda^k e^{-\lambda}}{k!}, \quad k = 0, 1, 2, \dots$$

$$E(X) = D(X) = \lambda$$

kde

- $\lambda > 0$ je průměrný počet událostí (intenzita),
- k je počet událostí,
- $e \approx 2,718$ je Eulerovo číslo.

Aproximace: pro velké n a malé p platí $\text{Bin}(n, p) \approx \text{Pois}(\lambda)$, kde $\lambda = np$.

Implementace v R

Funkce	Příklad použití	Vysvětlení
<code>dpois(k, lambda)</code>	<pre>dpois(2, lambda=4) # P(X=2) ppois(2, lambda=4) # P(X<=2) qpois(0.9, lambda=4) # kvantil rpois(100, lambda=4) # simulace</pre>	d/p/q/r = hustota / distribuční fce / kvantilová fce / simulace.

Implementace v Python

Funkce	Příklad použití	Vysvětlení
<code>stats.poisson.pmf(k, mu)</code>	<pre>from scipy.stats import poisson poisson.pmf(2, mu=4) # P(X=2) poisson.cdf(2, mu=4) # P(X<=2)</pre>	Poissonovo rozdělení z knihovny <code>scipy.stats</code> .

4.1.3 Hypergeometrické rozdělení $\text{Hyper}(N, K, n)$

Popisuje počet úspěchů při výběru n prvků **bez návratu** z populace N prvků, z nichž K je úspěšných. Rozdíl od binomického: pravděpodobnost úspěchu se mění s každým tahem.

$$P(X = k) = \frac{\binom{K}{k} \binom{N-K}{n-k}}{\binom{N}{n}}, \quad k = \max(0, n + K - N), \dots, \min(n, K)$$

$$E(X) = \frac{nK}{N}, \quad D(X) = \frac{nK}{N} \cdot \frac{N-K}{N} \cdot \frac{N-n}{N-1}$$

kde

- N je velikost populace,
- K je počet úspěchů v populaci,
- n je velikost výběru,
- k je počet úspěchů ve výběru.

Implementace v R

Funkce	Příklad použití	Vysvětlení
<code>dhyper(k, K, N-K, n)</code>	<pre># N=50, K=20 uspesnych, n=10 # vybranych dhyper(3, m=20, n=30, k=10) # P(# X=3) phyper(3, m=20, n=30, k=10) # P(# X<=3) # argumenty: m=K, n=N-K, k=vyber # n</pre>	d/p/q/r = hustota / distribuční fce / kvantilová fce / simulace.

Implementace v Python

Funkce	Příklad použití	Vysvětlení
<code>stats.hypergeom.pmf(k,M,n,N)</code>	<pre>from scipy.stats import hypergeom hypergeom.pmf(3, M=50, n=20, N =10) # M = velikost populace (N v # matematice) # n = pocet uspesnych v populaci # (K) # N = velikost vyberu (n v # matematice)</pre>	Hypergeometrické rozdělení z knihovny <code>scipy.stats</code> .

4.2 Spojitá rozdělení

4.2.1 Normální rozdělení $N(\mu, \sigma^2)$

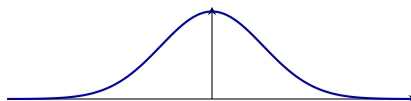
Nejdůležitější spojité rozdělení. Hustota pravděpodobnosti má tvar symetrické zvonové křivky kolem střední hodnoty μ .

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right), \quad x \in \mathbb{R}$$

$$E(X) = \mu, \quad D(X) = \sigma^2$$

kde

- μ je střední hodnota (poloha středu),
- σ^2 je rozptyl ($\sigma > 0$ je směrodatná odchylka),
- $e \approx 2,718$ je Eulerovo číslo, $\pi \approx 3,14159$.



Standardizace

Libovolnou normálně rozdělenou veličinu lze převést na standardní normální $Z \sim N(0, 1)$:

$$Z = \frac{X - \mu}{\sigma} \sim N(0, 1)$$

Standardní normální rozdělení má $E(Z) = 0$ a $D(Z) = 1$.

Pravidlo 6σ

Oblast $\pm 6\sigma$ kolem střední hodnoty pokrývá prakticky celou distribuci. Pravidlo se využívá v managementu kvality (metodologie *Six Sigma*):

$$P(\mu - 6\sigma \leq X \leq \mu + 6\sigma) \approx 0,9999999980$$

Procesy s variabilitou nepřesahující $\pm 6\sigma$ od cílové hodnoty produkují méně než 3,4 závad na milion příležitostí (DPMO).

Implementace v R

Funkce	Příklad použití	Vysvětlení
dnorm / pnorm / qnorm / rnorm	<pre>pnorm(185, mean=180, sd=7) # F(185) qnorm(0.975) # kvantil N(0,1) rnorm(100, mean=0, sd=1) # simulace # standardizace (ekvivalentni): pnorm(185, 180, 7) pnorm((185-180)/7)</pre>	d/p/q/r = hustota / distribuční fce / kvantilová fce / simulace.

Implementace v Python

Funkce	Příklad použití	Vysvětlení
stats.norm.cdf(x, loc, scale)	<pre>from scipy.stats import norm norm.cdf(185, loc=180, scale=7) # F(185) norm.ppf(0.975) # kvantil N(0,1) norm.rvs(size=100) # simulace</pre>	loc = μ , scale = σ .

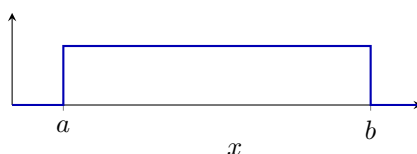
4.2.2 Rovnoměrné rozdělení $U(a, b)$

Každá hodnota v intervalu $[a, b]$ je stejně pravděpodobná. Hustota pravděpodobnosti je na celém intervalu konstantní.

$$f(x) = \frac{1}{b-a}, \quad x \in [a, b], \quad f(x) = 0 \text{ jinak}$$

$$E(X) = \frac{a+b}{2}, \quad D(X) = \frac{(b-a)^2}{12}$$

kde a je dolní mez a b je horní mez intervalu.



Implementace v R

Funkce	Příklad použití	Vysvětlení
dunif / punif / qunif / runif	<pre>dunif(0.5, min=0, max=1) # hustota f(0.5) punif(0.5, min=0, max=1) # P(X <=0.5) qunif(0.75, min=0, max=1) # kvantil runif(100, min=0, max=1) # simulace</pre>	min= a , max= b .

Implementace v Python

Funkce	Příklad použití	Vysvětlení
uniform.cdf(x, loc, scale)	<pre>from scipy.stats import uniform # loc=a, scale=b-a uniform.pdf(0.5, loc=0, scale=1) uniform.cdf(0.5, loc=0, scale=1) # P(X<=0.5) uniform.ppf(0.75, loc=0, scale =1) # kvantil</pre>	loc= a , scale= $b - a$.

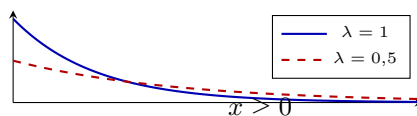
4.2.3 Exponenciální rozdělení $\text{Exp}(\lambda)$

Modeluje dobu čekání na první událost Poissonova procesu. Charakteristická vlastnost: **absence paměti** – $P(X > s + t \mid X > s) = P(X > t)$.

$$f(x) = \lambda e^{-\lambda x}, \quad x \geq 0$$

$$E(X) = \frac{1}{\lambda}, \quad D(X) = \frac{1}{\lambda^2}$$

kde $\lambda > 0$ je intenzita (průměrný počet událostí za jednotku času).



Implementace v R

Funkce	Příklad použití	Vysvětlení
dexp / pexp / qexp / rexp	<pre>dexp(1, rate=0.5) # hustota f(1) pro lambda=0.5 pexp(2, rate=0.5) # P(X<=2) qexp(0.9, rate=0.5) # kvantil rexp(100, rate=0.5) # simulace</pre>	rate= λ .

Implementace v Python

Funkce	Příklad použití	Vysvětlení
<code>expon.cdf(x, scale)</code>	<pre>from scipy.stats import expon # scale = 1/lambda expon.pdf(1, scale=2) # hustota pro lambda=0.5 expon.cdf(2, scale=2) # P(X<=2) expon.ppf(0.9, scale=2) # kvantil</pre>	$\text{scale} = 1/\lambda$.

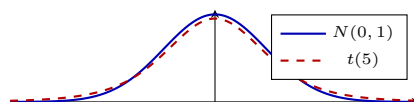
4.3 Odvozená rozdělení

4.3.1 Studentovo t-rozdělení $t(\nu)$

Vzniká jako podíl standardní normální veličiny a odmocniny veličiny χ^2 dělené svými stupni volnosti. Klíčové pro testování hypotéz a intervaly spolehlivosti, když **neznáme rozptyl** populace σ^2 .

$$E(X) = 0 \ (\nu > 1), \quad D(X) = \frac{\nu}{\nu - 2} \ (\nu > 2)$$

kde ν jsou stupně volnosti. Pro $\nu \rightarrow \infty$ přechází t-rozdělení v $N(0, 1)$.



Implementace v R

Funkce	Příklad použití	Vysvětlení
<code>dt / pt / qt / rt</code>	<pre>dt(0, df=5) # hustota f(0) pt(-1.96, df=19) # P(T<=t) qt(0.975, df=19) # krit. hodnota pro t-test/IS rt(100, df=10) # simulace</pre>	$\text{df} = \nu$ (stupně volnosti).

Implementace v Python

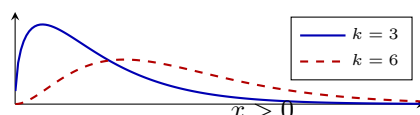
Funkce	Příklad použití	Vysvětlení
<code>t.pdf / cdf / ppf</code>	<pre>from scipy.stats import t t.pdf(0, df=5) # hustota f(0) t.cdf(-1.96, df=19) # P(T<=t) t.ppf(0.975, df=19) # krit. hodnota t.rvs(df=10, size=100) # simulace</pre>	$\text{df} = \nu$ (stupně volnosti).

4.3.2 Chí-kvadrát rozdělení $\chi^2(k)$

Vzniká jako součet kvadrátů k nezávislých standardně normálních veličin. Použití: test dobré shody, test nezávislosti v kontingenční tabulce.

$$E(X) = k, \quad D(X) = 2k$$

kde $k > 0$ jsou stupně volnosti. Rozdělení je pravostranně asymetrické; pro velká k se přibližuje normálnímu.



Implementace v R

Funkce	Příklad použití	Vysvětlení
dchisq / pchisq / qchisq / rchisq	<pre>dchisq(5, df=3) # hustota f(5) pchisq(7.81, df=3) # P(X<=7.81) qchisq(0.95, df=3) # krit. hodnota rchisq(100, df=3) # simulace</pre>	df = k (stupně volnosti).

Implementace v Python

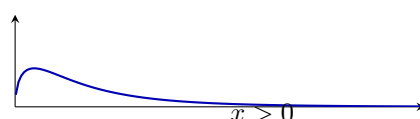
Funkce	Příklad použití	Vysvětlení
chi2.pdf / cdf / ppf	<pre>from scipy.stats import chi2 chi2.pdf(5, df=3) # hustota f(5) chi2.cdf(7.81, df=3) # P(X <=7.81) chi2.ppf(0.95, df=3) # krit. hodnota chi2.rvs(df=3, size=100) # simulace</pre>	df = k (stupně volnosti).

4.3.3 F-rozdělení $F(k_1, k_2)$

Vzniká jako podíl dvou nezávislých veličin χ^2 , každé dělené svými stupni volnosti. Použití: ANOVA, F-test celkové platnosti regresního modelu.

$$E(X) = \frac{k_2}{k_2 - 2} \quad (k_2 > 2), \quad D(X) = \frac{2k_2^2(k_1 + k_2 - 2)}{k_1(k_2 - 2)^2(k_2 - 4)} \quad (k_2 > 4)$$

kde k_1 jsou stupně volnosti čitatele a k_2 jmenovatele.



Implementace v R

Funkce	Příklad použití	Vysvětlení
df / pf / qf / rf	<pre>df(2, df1=3, df2=10) # hustota f (2) pf(3.71, df1=3, df2=10) # P(F <=3.71) qf(0.95, df1=3, df2=10) # krit. hodnota rf(100, df1=3, df2=10) # simulace # pozor: df() koliduje s data. frame # pouzit: stats::df()</pre>	df1= k_1 (čitatel), df2= k_2 (jmenovatel).

Implementace v Python

Funkce	Příklad použití	Vysvětlení
f.pdf / cdf / ppf	<pre>from scipy.stats import f f.pdf(2, dfn=3, dfd=10) # hustota f(2) f.cdf(3.71, dfn=3, dfd=10) # P(F <=3.71) f.ppf(0.95, dfn=3, dfd=10) # krit. hodnota f.rvs(dfn=3, dfd=10, size=100) # simulace</pre>	dfn= k_1 (čitatel), dfd= k_2 (jmenovatel).

5 Testování hypotéz

5.1 Chyby testování hypotéz

Rozhodnutí o zamítnutí H_0 může být správné nebo chybné. **Hladina významnosti** $\alpha = P(\text{chyba I. typu})$ se volí předem (typicky 0,05 nebo 0,01).

	H_0 se nezamítá	H_0 se zamítá
H_0 je pravdivá	Správné rozhodnutí ($1 - \alpha$)	Chyba I. typu (α)
H_0 není pravdivá	Chyba II. typu (β)	Správné rozhodnutí ($1 - \beta$) = síla testu

- α (**chyba I. typu**): zamítneme H_0 , přestože platí. Volíme sami jako hladinu významnosti.
- β (**chyba II. typu**): nezamítneme H_0 , přestože neplatí. Závisí na velikosti výběru a síle efektu.
- $1 - \beta$ (**síla testu**): pravděpodobnost správného zamítnutí nepravdivé H_0 .
- Snížení α zpravidla zvyšuje β (při stejném n).

5.2 Bodový odhad

Bodový odhad nahrazuje neznámý parametr populace jednou hodnotou vypočtenou z výběru. Výběrový průměr $\bar{x}$ a výběrový rozptyl s^2 jsou **nestranné** odhady (jejich střední hodnota se rovná odhadovanému parametru).

Parametr	Odhad (vzorec)	Implementace v R
Střední hodnota μ	$\hat{\mu} = \bar{x} = \frac{1}{n} \sum x_i$	<code>mean(x)</code>
Rozptyl σ^2	$\hat{\sigma}^2 = s^2 = \frac{\sum (x_i - \bar{x})^2}{n - 1}$	<code>var(x)</code>
Podíl π	$\hat{p} = x/n$	<code>sum(x == k) / length(x)</code>

5.3 Interval spolehlivosti a velikost výběru

5.3.1 Interval pro průměr (μ)

Odhadujeme skutečnou střední hodnotu populace μ na základě výběrového průměru $\bar{x}$.

- **Neznámý σ^2 (nebo $n < 30$):** $\bar{x} \pm t_{1-\alpha/2}(n-1) \cdot \frac{s}{\sqrt{n}}$
- **Známý σ^2 (nebo $n \geq 30$):** $\bar{x} \pm z_{1-\alpha/2} \cdot \frac{\sigma}{\sqrt{n}}$

Pro **oboustranný** interval používáme kvantil $1 - \alpha/2$, pro **jednostranný** $1 - \alpha$.

Implementace v R

Funkce	Příklad použití	Vysvětlení
<code>t.test(x)\$conf.int</code>	<pre>x <- c(163,175,177,165,171,174) t.test(x, conf.level = 0.95) \$conf.int</pre>	Vrací 95% IS pro μ (neznámý σ^2).
Ručně (t-rozdělení)	<pre>xbar <- mean(x); s <- sd(x) n <- length(x); alpha <- 0.05 t_kr <- qt(1 - alpha/2, df = n - 1) c(xbar - t_kr*s/sqrt(n), xbar + t_kr*s/sqrt(n))</pre>	Ruční výpočet IS s t-rozdělením.
Ručně (Z, známý σ)	<pre>sigma <- 7 z_kr <- qnorm(1 - alpha/2) c(xbar - z_kr*sigma/sqrt(n), xbar + z_kr*sigma/sqrt(n))</pre>	Ruční výpočet IS s normálním rozdělením (velký výběr nebo známý σ).

Implementace v Python

Funkce	Příklad použití	Vysvětlení
Kritická hodnota	<pre>crit = t.ppf(1 - alpha/2, df=n-1)</pre>	Kvantil pro t-interval (neznámé σ). Pro velké n lze nahradit <code>norm.ppf</code> .
Mezní chyba	<pre>err = crit * (s / np.sqrt(n))</pre>	Výpočet odchylky; s je výběrová směrodatná odchylka.
Meze intervalu	<pre>lower, upper = xbar - err, xbar + err</pre>	Parametr leží v intervalu s pravděpodobností $1 - \alpha$.

5.3.2 Interval pro podíl (π)

Odhadujeme skutečný podíl π z výběrové relativní četnosti $\hat{p} = x/n$. Vždy použijeme normální rozdělení (Z-kvantil).

$$\hat{p} \pm z_{1-\alpha/2} \cdot \sqrt{\frac{\hat{p}(1-\hat{p})}{n}}$$

Implementace v R

Funkce	Příklad použití	Vysvětlení
<code>prop.test(x, n)\$conf.int</code>	<pre>prop.test(x=110, n=200, conf.level=0.95)\$conf.int</pre>	Wilsonův IS pro proporci (vestavěná funkce).
Ručně (Waldův IS)	<pre>x_vol <- 110; n_vol <- 200 p <- x_vol / n_vol z_kr <- qnorm(0.975) se <- sqrt(p*(1-p)/n_vol) c(p - z_kr*se, p + z_kr*se)</pre>	Waldův IS – jednoduchá normální aproximace.

Implementace v Python

Funkce	Příklad použití	Vysvětlení
Kritická hodnota	<pre>z_crit = norm.ppf(1 - alpha / 2)</pre>	Pro podíl se vždy používá normální rozdělení (Z-kvantil).
Mezní chyba	<pre>err = z_crit * np.sqrt(p * (1-p) / n)</pre>	Výpočet odchylky pro proporci $\hat{p}$.
Meze intervalu	<pre>lower, upper = p - err, p + err</pre>	Parametr leží v intervalu s pravděpodobností $1 - \alpha$.

5.3.3 Potřebná velikost výběru (n)

Minimální rozsah výběru, aby se odhad nemýlil o více než Δ . Výsledek **vždy zaokrouhlit nahoru**.

$$n \geq \left(\frac{z_{1-\alpha/2} \cdot \sigma}{\Delta} \right)^2 \quad (\text{průměr}), \quad n \geq \hat{p}(1 - \hat{p}) \cdot \left(\frac{z_{1-\alpha/2}}{\Delta} \right)^2 \quad (\text{podíl})$$

Implementace v R

Funkce	Příklad použití	Vysvětlení
Minimální n (průměr)	<pre>alpha <- 0.05; sigma <- 10 delta <- 2 # pozadovana presnost z_kr <- qnorm(1 - alpha/2) ceiling((z_kr * sigma / delta)^2)</pre>	Pro průměr. Δ je cílová přesnost. Výsledek vždy zaokrouhli nahoru.
Minimální n (podíl)	<pre>p <- 0.5; delta <- 0.05 # +-5 p. b. ceiling(p*(1-p)*(z_kr/delta)^2)</pre>	Pro podíl. Používá se pro plánování reprezentativního výzkumu.

Implementace v Python

Funkce	Příklad použití	Vysvětlení
Minimální n (průměr)	<pre>n = math.ceil((z_crit * sigma / delta)**2)</pre>	Pro průměr. Δ je cílová přesnost. Výsledek vždy zaokrouhli nahoru.
Minimální n (podíl)	<pre>n = math.ceil(p*(1-p) * (z_crit/ delta)**2)</pre>	Pro podíl.

5.4 Jednovýběrový t-test (střední hodnota)

$H_0: \mu = \mu_0$. Testové kritérium má t-rozdělení s $n - 1$ stupni volnosti.

$$T = \frac{\bar{x} - \mu_0}{s/\sqrt{n}} \sim t(n - 1)$$

Typ testu	Alternativa H_1	p-hodnota v R
Levostranný	$\mu < \mu_0$	<code>pt(T, df=n-1)</code>
Pravostranný	$\mu > \mu_0$	<code>1 - pt(T, df=n-1)</code>
Oboustranný	$\mu \neq \mu_0$	<code>2*(1 - pt(abs(T), df=n-1))</code>

Implementace v R

Funkce	Příklad použití	Vysvětlení
<code>t.test(x, mu=mu0, ...)</code>	<pre>t.test(x, mu = 25000, alternative = "less") # "greater" / "two.sided"</pre>	Automatický t-test z vektoru dat.
Ručně	<pre>n <- 49; xbar <- 23500; alpha <- 0.05 mu0 <- 25000; s <- 5500 T <- (xbar - mu0)/(s/sqrt(n)) p_val <- pt(T, df = n - 1) t_kr <- qt(alpha, df = n - 1)</pre>	Ruční výpočet (jednostranný levostranný).

Implementace v Python

Funkce	Příklad použití	Vysvětlení
<code>stats.ttest_1samp(data, mu0)</code>	<code>stats.ttest_1samp(data, popmean=mu0)</code>	Porovnání průměru vzorku s pevnou hodnotou μ_0 .
Ručně	<pre>from scipy.stats import t as t_dist n=49; xbar=23500; s=5500 mu0=25000; alpha=0.05 T = (xbar - mu0) / (s / np.sqrt(n)) p_val = t_dist.cdf(T, df=n-1) t_kr = t_dist.ppf(alpha, df=n-1)</pre>	Ruční výpočet (jednostranný levostranný); pro pravostranný: <code>1-cdf(T,...)</code> ; pro oboustranný: <code>2*(1-cdf(abs(T),...))</code> .

5.5 Dvouvýběrový t-test (Welchův)

$H_0: \mu_1 = \mu_2$. Welchova aproximace stupňů volnosti (předpokládáme různé rozptyly).

$$T = \frac{\bar{x}_1 - \bar{x}_2}{\sqrt{s_1^2/n_1 + s_2^2/n_2}}$$

Implementace v R

Funkce	Příklad použití	Vysvětlení
<code>t.test(x1, x2, var.equal=FALSE)</code>	<code>t.test(x1, x2, var.equal = FALSE, alternative = "two.sided")</code>	Welchův test (různé rozptyly).
Ručně (ze souhrnných statistik)	<pre>n1<-38; x1<-5.3875; s1_2<-0.2698 n2<-35; x2<-4.7; s2_2<-0.24 T <- (x1-x2)/sqrt(s1_2/n1+s2_2/n2) df_w <- (s1_2/n1+s2_2/n2)^2 / ((s1_2/n1)^2/(n1-1)+ (s2_2/n2)^2/(n2-1)) p_val <- 2*(1-pt(abs(T), df=df_w))</pre>	Welch-Satterthwaiteovy stupně volnosti (s_1^2, s_2^2 jsou rozptyly).

Implementace v Python

Funkce	Příklad použití	Vysvětlení
<code>stats.ttest_ind(A, B)</code>	<code>stats.ttest_ind(A, B, equal_var=False)</code>	Porovnání průměrů dvou nezávislých skupin (Welchův test).

5.6 Párový t-test

$H_0: \mu_d = 0$, kde $d_i = x_{2i} - x_{1i}$ jsou párové rozdíly. Aplikuje jednovýběrový t-test na rozdíly.

Implementace v R

Funkce	Příklad použití	Vysvětlení
<code>t.test(pred, po, paired=TRUE)</code>	<pre>t.test(pred, po, paired = TRUE, alternative = "two.sided")</pre>	Porovnání stejné skupiny ve dvou různých situacích (např. před a po zásahu).

Implementace v Python

Funkce	Příklad použití	Vysvětlení
<code>stats.ttest_rel(po, pred)</code>	<pre>stats.ttest_rel(po, pred)</pre>	Porovnání stejné skupiny ve dvou různých situacích.

5.7 Test proporce (Z-test)

$H_0: \pi = \pi_0$. Testové kritérium:

$$Z = \frac{\hat{p} - \pi_0}{\sqrt{\pi_0(1 - \pi_0)/n}} \sim N(0, 1)$$

Implementace v R

Funkce	Příklad použití	Vysvětlení
<code>prop.test(x, n, p=p0, ...)</code>	<pre>prop.test(x=90, n=150, p=0.5, alternative="two.sided", correct=FALSE)</pre>	Bez Yatesovy korekce odpovídá ručnímu Z-testu.
Ručně	<pre>p0 <- 0.5; n <- 150; orel <- 90 phat <- orel/n Z <- (phat-p0)/sqrt(p0*(1-p0)/n) z_kr <- qnorm(1 - 0.01/2) p_val <- 2*(1 - pnorm(abs(Z)))</pre>	Oboustranný Z-test symetrie mince ($\alpha = 0,01$).

Implementace v Python

Funkce	Příklad použití	Vysvětlení
<code>proportions_ztest</code>	<pre>from statsmodels.stats. proportion import proportions_ztest z_stat, p_val = proportions_ztest(count=90, nobs=150, value=0.5, alternative='two- sided')</pre>	Z-test proporce. <code>count</code> = počet úspěchů, <code>nobs</code> = rozsah výběru, <code>value</code> = π_0 . Vrací (Z, p) .

5.8 Jednocestná analýza rozptylu (ANOVA)

H_0 : $\mu_1 = \mu_2 = \dots = \mu_k$, H_1 : alespoň dva průměry se liší.

Test je vždy **pravostranný**: zamítáme H_0 , pokud $F \geq F_{1-\alpha}(k-1, n-k)$.

$$F = \frac{S_m/(k-1)}{S_v/(n-k)} \sim F(k-1, n-k), \quad S_m = \sum_{j=1}^k n_j (\bar{x}_j - \bar{x})^2, \quad S_v = \sum_{j=1}^k \sum_{i=1}^{n_j} (x_{ij} - \bar{x}_j)^2$$

Poměr determinace η^2 udává podíl variability vysvětlené skupinovým faktorem ($S_c = S_m + S_v$ je celkový součet čtverců):

$$\eta^2 = \frac{S_m}{S_c} = \frac{S_m}{S_m + S_v} \in [0, 1]$$

Implementace v R

Funkce	Příklad použití	Vysvětlení
aov(y ~ skupina)	<pre>y <- c(91,81,74,57, 83,72,63,47, 71,69,58,40) skupina <- factor(rep(c("A","B", "","C"), each=4)) m <- aov(y ~ skupina) summary(m)</pre>	Automatická ANOVA s tabulkou výsledků.
Skupinové průměry	<pre>tapply(y, skupina, mean)</pre>	Průměry v každé skupině.
Kritická hodnota F	<pre>k <- nlevels(skupina); n <- length(y) qf(0.95, df1 = k-1, df2 = n-k)</pre>	Kvantil F-rozdělení pro $\alpha = 0,05$.
Ručně (SS)	<pre>grand <- mean(y) SS_m <- sum(tapply(y,skupina, function(g) length(g)*(mean(g)- grand)^2)) SS_v <- sum(tapply(y,skupina, function(g) sum((g-mean(g))^2))) F_stat <- (SS_m/(k-1))/(SS_v/(n- k)) eta2 <- SS_m / (SS_m + SS_v)</pre>	Ruční výpočet součtů čtverců; $\eta^2 = S_m/S_c$ je poměr determinace.

Implementace v Python

Funkce	Příklad použití	Vysvětlení
<code>stats.f_oneway(A, B, C)</code>	<code>stats.f_oneway(A, B, C)</code>	Testování shody průměrů u tří a více skupin.
Poměr determinace η^2	<pre>import numpy as np groups = [A, B, C] y_all = np.concatenate(groups) grand = y_all.mean() SS_m = sum(len(g)*(g.mean()-grand)**2 for g in groups) SS_v = sum(((g - g.mean())**2).sum() for g in groups) eta2 = SS_m / (SS_m + SS_v)</pre>	Poměr determinace $\eta^2 = S_m/S_c$; hodnota 0–1, jak velkou část variability vysvětluje faktor skupiny.

5.9 Chí-kvadrát test dobré shody

H_0 : výběr odpovídá předpokládanému rozdělení.

$$G = \sum_j \frac{(n_j - e_j)^2}{e_j} \sim \chi^2(k-1), \quad e_j = n \cdot \pi_{0,j}$$

Implementace v R

Funkce	Příklad použití	Vysvětlení
<code>chisq.test(emp, p=p_teor)</code>	<pre>emp <- c(A=82, B=38, AB=26, 0=69) p0 <- c(A=0.41, B=0.14, AB=0.07, 0=0.38) chisq.test(emp, p = p0)</pre>	Automatický test dobré shody. Vektor <code>p0</code> musí mít součet 1.
Ručně	<pre>teor <- sum(emp) * p0 G <- sum((emp - teor)^2 / teor) p_val <- 1 - pchisq(G, df=length(emp)-1) chi_kr <- qchisq(0.99, df=length(emp)-1)</pre>	Ruční výpočet testové statistiky a p-hodnoty.

Implementace v Python

Funkce	Příklad použití	Vysvětlení
<code>stats.chisquare(O, f_exp=E)</code>	<code>stats.chisquare(O, f_exp=E)</code>	Test dobré shody. <code>O</code> jsou pozorované, <code>E</code> očekávané četnosti.

5.10 Chí-kvadrát test nezávislosti

H_0 : řádkový a sloupcový znak jsou nezávislé. Kontingenční tabulka $r \times s$: $n_{i.} = \sum_j n_{ij}$ (řádkové součty), $n_{.j} = \sum_i n_{ij}$ (sloupcové součty).

$$n'_{ij} = \frac{n_{i.} n_{.j}}{n}, \quad n'_{ij} \geq 5, \quad G = \sum_{i=1}^r \sum_{j=1}^s \frac{(n_{ij} - n'_{ij})^2}{n'_{ij}} \sim \chi^2((r-1)(s-1))$$

Pro tabulku 2×2 :

$$G = \frac{n(n_{11}n_{22} - n_{12}n_{21})^2}{n_{1.} n_{2.} n_{.1} n_{.2}}$$

Síla závislosti ($m = \min(r, s)$):

$$C = \sqrt{\frac{G}{G+n}} \in \left[0, \sqrt{\frac{m-1}{m}}\right) \quad V = \sqrt{\frac{G}{n(m-1)}} \in [0, 1]$$

Pearsonovo C nikdy nedosáhne hodnoty 1 – ve jmenovateli je vždy $G+n > G$, takže zlomek je přísně menší než 1. Maximum závisí na rozměrech tabulky. **Cramérovo** V je upraveno tak, aby při dokonalé závislosti dosahovalo 1, a je proto vhodné pro porovnání tabulek různých rozměrů.

Interpretace V : $< 0,1$ zanedbatelná · $0,1-0,3$ slabá · $0,3-0,5$ středně silná · $> 0,5$ silná.

Implementace v R

Funkce	Příklad použití	Vysvětlení
<code>chisq.test(tabulka)</code>	<pre>kt <- matrix(c(1768,807,189,47, 946,1387,746,53, 115,438,288,16), nrow=3, byrow=TRUE) res <- chisq.test(kt) res\$p.value # p-hodnota res\$expected # teoretické četnosti</pre>	Test nezávislosti z kontingenční tabulky.
Cramérovo V	<pre>G <- as.numeric(res\$statistic) n_celk <- sum(kt) r <- nrow(kt); s_kt <- ncol(kt) V <- sqrt(G / (n_celk*(min(r, s_kt)-1))) C <- sqrt(G / (G + n_celk))</pre>	Síla závislosti: $V \in [0, 1]$. Pearsonův C závisí na rozměrech tabulky.

Implementace v Python

Funkce	Příklad použití	Vysvětlení
<code>stats.chi2_contingency(tabl</code>	<pre>chi2, p, df, exp = stats. chi2_contingency(table)</pre>	Vrací $(\chi^2, p, df, \text{teoretické četnosti})$.

5.11 Stupně volnosti – přehled

Zlaté pravidlo: stupně volnosti = počet pozorování – počet odhadovaných parametrů.

T-testy

Test	df	Poznámka
Jednovýběrový t-test	$df = n - 1$	n = počet pozorování
Párový t-test	$df = n_{\text{párů}} - 1$	n = počet párů
Dvouvýběrový t-test (shoda rozptylů)	$df = n + m - 2$	n, m = velikosti skupin
Dvouvýběrový t-test (Welchův)	$df \approx \text{Welch}$	složitý vzorec – spočítá program

F-testy

Test	df_1	df_2	Poznámka
Test shody rozptylů	$n - 1$	$m - 1$	dva df – jeden pro každý soubor
ANOVA (F-test)	$k - 1$	$n - k$	k = počet skupin

Chí-kvadrát testy

Test	df
Test dobré shody	$df = k - 1$ (k = počet kategorií)
Test nezávislosti (konting. tabulka)	$df = (r - 1)(s - 1)$ (r řádků, s sloupců)

6 Analýza závislosti

6.1 Korelační analýza

6.1.1 Pearsonův korelační koeficient

Měří sílu **lineární** závislosti dvou spojitých proměnných.

$$r = \frac{s_{xy}}{s_x \cdot s_y} = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2 \cdot \sum_{i=1}^n (y_i - \bar{y})^2}}, \quad r \in [-1, 1]$$

kde $s_{xy} = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})$ je výběrová kovariance.

Znaménko: r má vždy stejné znaménko jako s_{xy} (dělíme kladnými s_x, s_y). Kovariance, korelační koeficient i směrnice regresní přímky mají proto vždy shodné znaménko.

Test $H_0: \rho = 0$: $T = r\sqrt{(n-2)/(1-r^2)} \sim t(n-2)$.

Interpretace $|r|$: $< 0,3$ slabá $\cdot$ $0,3$ – $0,7$ střední $\cdot$ $> 0,7$ silná závislost. Znaménko udává směr.

Implementace v R

Funkce	Příklad použití	Vysvětlení
<code>cor(x, y, method="pearson")</code>	<pre>x <- c(1,2,0,5,3,4,2,6,1,3) y <- c(0,1,0,4,2,3,1,5,0,2) cor(x, y, method = "pearson")</pre>	Hodnota korelačního koeficientu $r \in [-1, 1]$.
<code>cor.test(x, y)</code>	<pre>cor.test(x, y, method = "pearson")</pre>	Test $H_0: \rho = 0$; vrací p -hodnotu, T a IS pro ρ .
Ručně	<pre>s_xy <- cov(x, y) r <- s_xy / (sd(x) * sd(y)) n <- length(x) T <- r * sqrt((n-2)/(1-r^2)) p_val <- 2*(1 - pt(abs(T), df=n-2))</pre>	Ruční výpočet přes kovarianci; p -hodnota pro oboustranný test $H_0: \rho = 0$.

Implementace v Python

Funkce	Příklad použití	Vysvětlení
<code>stats.pearsonr(x, y)</code>	<pre>r, p = stats.pearsonr(x, y)</pre>	Vrací korelační koeficient r a p -hodnotu testu $H_0: \rho = 0$.
Ručně	<pre>import numpy as np s_xy = np.cov(x, y, ddof=1)[0,1] r = s_xy / (np.std(x, ddof=1) * np.std(y, ddof=1)) n = len(x) T = r * np.sqrt((n-2)/(1-r**2))</pre>	Ruční výpočet přes kovarianci; <code>np.cov</code> vrací kovarianční matici, <code>[0,1]</code> vybere s_{xy} .

6.1.2 Koeficient determinace R^2

Udává, jaký podíl variability Y je vysvětlen lineární závislostí na X . Pro jednoduchou lineární regresi platí $R^2 = r^2$ (čtverec Pearsonova korelačního koeficientu).

$$R^2 = r^2 \in [0, 1]$$

Příklad: $r = 0,9 \Rightarrow R^2 = 0,81$ – model vysvětluje 81 % variability Y . Záporná korelace dává stejný R^2 : $r = -0,9 \Rightarrow R^2 = 0,81$.

Implementace v R

Funkce	Příklad použití	Vysvětlení
R^2 z korelace	<pre>R2 <- cor(x, y)^2</pre>	Koeficient determinace jako čtverec Pearsonova r .
R^2 z regresního modelu	<pre>m <- lm(y ~ x) R2 <- summary(m)\$r.squared</pre>	<code>summary(lm())</code> vrací R^2 přímo (<code>r.squared</code>).

Implementace v Python

Funkce	Příklad použití	Vysvětlení
R^2 z korelace	<pre>from scipy.stats import pearsonr r, _ = pearsonr(x, y) R2 = r**2</pre>	Koeficient determinace jako čtverec Pearsonova r .
R^2 z regresního modelu	<pre>from scipy.stats import linregress res = linregress(x, y) R2 = res.rvalue**2</pre>	<code>rvalue</code> je Pearsonovo r ; jeho čtverec je R^2 .

6.1.3 Spearmanův korelační koeficient

Měří sílu **monotónní** závislosti (neparametrický, odolný vůči odlehlým hodnotám). Počítá se jako Pearsonův koeficient aplikovaný na pořadí hodnot.

Implementace v R

Funkce	Příklad použití	Vysvětlení
<code>cor(x, y, method="spearman")</code>	<code>cor(x, y, method = "spearman")</code>	Hodnota Spearmanova koeficientu r_s .
<code>cor.test(x, y, method="spearman")</code>	<code>cor.test(x, y, method = "spearman")</code>	Test $H_0: \rho_s = 0$; vrací p -hodnotu a testovou statistiku.

Implementace v Python

Funkce	Příklad použití	Vysvětlení
<code>stats.spearmanr(x, y)</code>	<code>rho, p = stats.spearmanr(x, y)</code>	Vrací Spearmanův r_s a p -hodnotu testu $H_0: \rho_s = 0$.

6.2 Jednoduchá lineární regrese

6.2.1 Odhad koeficientů (MNČ)

Model: $\hat{y} = b_0 + b_1x$. Koeficienty minimalizují součet čtverců reziduí $Q = \sum e_i^2$.

$$b_1 = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2}, \quad b_0 = \bar{y} - b_1\bar{x}$$

Implementace v R

Funkce	Příklad použití	Vysvětlení
<code>lm(y ~ x)</code>	<pre>m <- lm(cena ~ naklady) summary(m) # koef., R^2, testy coef(m) # b0 a b1 fitted(m) # vyrovnané hodnoty residuals(m) # rezidua e_i</pre>	Fituje lineární model; <code>summary</code> vrací koeficienty, R^2 , t-testy i F-test automaticky.
Koeficienty ručně	<pre>x_b <- mean(x); y_b <- mean(y) b1 <- sum((x-x_b)*(y-y_b)) / sum((x-x_b)^2) b0 <- y_b - b1 * x_b</pre>	Ruční výpočet metodou nejmenších čtverců (minimalizace $Q = \sum e_i^2$).

Implementace v Python

Funkce	Příklad použití	Vysvětlení
<code>stats.linregress(x, y)</code>	<pre>res = stats.linregress(x, y)</pre>	Vrací <i>slope</i> (b_1), <i>intercept</i> (b_0), r , p -hodnotu a <code>stderr</code> (základní regrese).
<code>smf.ols(f, data)</code>	<pre>import statsmodels.formula.api as smf m = smf.ols('y ~ x', data=df).fit() print(m.summary()) # Komplexní výstup</pre>	Fituje model pomocí vzorce (stejná syntaxe jako R). Funkce <code>summary()</code> vrací koeficienty, R^2 , i testy.
Vícenásobná regrese	<pre># Model s více prediktory m2 = smf.ols('y ~ x1 + x2 + x3', data=df).fit() print(m2.summary()) # Upravený R^2 pro porovnání modelu print(m2.rsquared_adj)</pre>	<code>summary()</code> vypíše tabulku, kde Adj. R-squared použijeme k porovnání s jednodušším modelem (penalizuje za zbytečné parametry).

6.2.2 Součty čtverců a koeficient determinace R^2

$$S_Y = S_T + S_R, \quad R^2 = \frac{S_T}{S_Y} \in [0, 1], \quad R_{ADJ}^2 = 1 - (1 - R^2) \frac{n-1}{n-p}$$

kde $S_Y = \sum (y_i - \bar{y})^2$ (celkový), $S_T = \sum (\hat{y}_i - \bar{y})^2$ (teoretický), $S_R = \sum (y_i - \hat{y}_i)^2$ (reziduální), p = počet odhadovaných parametrů (pro jednoduchou regresi $p = 2$).

$$\text{MSE} = \frac{S_R}{n} = \frac{\sum (y_i - \hat{y}_i)^2}{n}, \quad \text{RMSE} = \sqrt{\text{MSE}}$$

MSE (střední čtvercová chyba) měří průměrnou kvadratickou odchylku předpovědi; RMSE je ve stejných jednotkách jako Y .

Implementace v R

Funkce	Příklad použití	Vysvětlení
Součty čtverců a R^2	<pre>SS_tot <- sum((y - mean(y))^2) SS_reg <- sum((fitted(m)-mean(y))^2) SS_res <- sum(residuals(m)^2) R2 <- SS_reg / SS_tot summary(m)\$r.squared # zkratka</pre>	$S_Y = S_T + S_R$; $R^2 = S_T/S_Y$ – podíl rozptylu Y vysvětlený modelem.
MSE a RMSE	<pre>MSE <- mean(residuals(m)^2) RMSE <- sqrt(MSE)</pre>	<code>residuals(m)</code> vrací $e_i = y_i - \hat{y}_i$; průměr jejich čtverců je MSE.

Implementace v Python

Funkce	Příklad použití	Vysvětlení
R^2	<pre>r_squared = res.rvalue**2</pre>	R^2 jako čtverec korelačního koeficientu (z <code>stats.linregress</code>).
MSE a RMSE	<pre>import numpy as np e = y - res.intercept - res.slope*x MSE = np.mean(e**2) RMSE = np.sqrt(MSE)</pre>	Rezidua ručně z parametrů <code>linregress</code> ; nebo <code>sklearn.metrics.mean_squared_error(y, y_hat)</code> .

6.2.3 t-test a F-test platnosti modelu

t-test pro b_1 : $H_0: \beta_1 = 0$; $T = b_1/s_{b_1} \sim t(n-2)$.

F-test modelu: $H_0: \beta_1 = 0$; $F = S_T/(S_R/(n-2)) \sim F(1; n-2)$.

Implementace v R

Funkce	Příklad použití	Vysvětlení
t-test pro b_1	<pre>n <- length(x) se_b1 <- sqrt(SS_res/(n-2) / sum((x-mean(x))^2)) t_stat <- b1 / se_b1 qt(0.975, df=n-2) # krit. hodnota 2*(1-pt(abs(t_stat), df=n-2)) # p-val</pre>	$H_0: \beta_1 = 0$; zamítáme při $ T \geq t_{1-\alpha/2}(n-2)$.
F-test modelu	<pre>F_stat <- (SS_reg/1)/(SS_res/(n-2)) qf(0.95, df1=1, df2=n-2)</pre>	Pro jednoduchou regresi ekvivalentní t-testu. Výsledek v <code>summary(m)</code> .

Implementace v Python

Funkce	Příklad použití	Vysvětlení
p-hodnota pro b_1	<pre>res.pvalue</pre>	p-hodnota testu $H_0: \beta_1 = 0$ (z <code>stats.linregress</code>).

6.2.4 Predikce

Bodová predikce střední hodnoty Y pro novou hodnotu x^* : $\hat{y}^* = b_0 + b_1x^*$.

Implementace v R

Funkce	Příklad použití	Vysvětlení
<code>predict(m, newdata=...)</code>	<pre>predict(m, newdata=data.frame(naklady=80)) predict(m, newdata=data.frame(naklady=80), interval="confidence")</pre>	Bodová predikce i IS pro střední hodnotu Y při nové hodnotě X .

Implementace v Python

Funkce	Příklad použití	Vysvětlení
Predikce (<code>scipy</code>)	<pre>y_hat = res.intercept + res. slope * x_new</pre>	Bodová predikce ručně z výsledků <code>stats.linregress</code> .
Predikce (<code>statsmodels</code>)	<pre>pred = m.get_prediction(nove_data) pred.summary_frame() # Obsahuje intervaly</pre>	Vrátí <code>DataFrame</code> obsahující bodovou predikci i interval spolehlivosti pro nová data.

7 Časové řady

7.1 Základní charakteristiky

7.1.1 Absolutní přírůstky a tempa růstu

1. difference: $\Delta y_t = y_t - y_{t-1}$

Průměrná 1. difference: $\bar{\Delta} = \frac{1}{n-1} \sum_{t=2}^n \Delta_t = \frac{y_n - y_1}{n-1}$

Tempo růstu: $k_t = y_t / y_{t-1}$

Průměrné tempo: $\bar{k} = \sqrt[n-1]{y_n / y_1}$

Implementace v R

Funkce	Příklad použití	Vysvětlení
<code>diff(y)</code>	<pre>y <- c (5274,5288,5297,5318,5338,5356) diff(y) # 1. difference diff(diff(y)) # 2. difference</pre>	Absolutní přírůstky první a druhé řádu.
Tempo růstu	<pre>n <- length(y) tempo <- y[-1] / y[-n] prum_tempo <- (y[n]/y[1])^(1/(n-1))</pre>	Mezičasové indexy k_t a průměrné tempo růstu (geometrický průměr).

Implementace v Python

Funkce	Příklad použití	Vysvětlení
<code>np.diff(y)</code>	<code>np.diff(y)</code>	Absolutní přírůstky (1. difference).
Tempo růstu	<code>y[1:] / y[:-1]</code>	Mezičasové indexy k_t pro NumPy pole <code>y</code> .

7.1.2 Průměr časové řady

Intervalová časová řada – prostý průměr:

$$\bar{y} = \frac{1}{n} \sum_{t=1}^n y_t$$

Okamžiková časová řada – chronologický průměr (stejně dlouhé intervaly):

$$\bar{y} = \frac{\frac{1}{2}y_1 + y_2 + \dots + y_{n-1} + \frac{1}{2}y_n}{n-1}$$

Pro nesterajně dlouhé intervaly (vážený chronologický průměr, délky intervalů $d_1, \dots, d_{n-1}$):

$$\bar{y} = \frac{\frac{y_1 + y_2}{2}d_1 + \frac{y_2 + y_3}{2}d_2 + \dots + \frac{y_{n-1} + y_n}{2}d_{n-1}}{d_1 + d_2 + \dots + d_{n-1}}$$

Implementace v R

Funkce	Příklad použití	Vysvětlení
Prostý průměr (intervalová)	<pre>mean(y)</pre>	Průměr intervalové časové řady.
Chronologický průměr (okamžiková)	<pre>n <- length(y) (y[1]/2 + sum(y[2:(n-1)]) + y[n]/2) / (n-1)</pre>	Průměr okamžikové časové řady; krajní hodnoty mají váhu $\frac{1}{2}$.

Implementace v Python

Funkce	Příklad použití	Vysvětlení
Prostý průměr (intervalová)	<pre>np.mean(y)</pre>	Průměr intervalové časové řady.
Chronologický průměr (okamžiková)	<pre>(y[0]/2 + np.sum(y[1:-1]) + y [-1]/2) / (len(y)-1)</pre>	Průměr okamžikové časové řady.

7.2 Řetězové a bazické indexy

Řetězový index $I_{t/t-1}$ (= koeficient růstu k_t): poměr k *předchozímu* období. **Bazický index** $I_{t/1}$: poměr k *základnímu* období (období 1). Oba výpočty jsou totožné – liší se pouze volbou základu srovnání.

Řetězový index / koeficient růstu:

$$k_t = I_{t/t-1} = \frac{y_t}{y_{t-1}} \quad \bar{k} = \sqrt[n-1]{k_2 \cdot k_3 \cdots k_n} = \sqrt[n-1]{\frac{y_n}{y_1}} \quad (\text{průměrné tempo růstu})$$

Průměrné tempo růstu $\bar{k}$ je **geometrický průměr** koeficientů růstu k_t .

Bazický index (rozklad přes řetězové indexy a zpětný přepočet):

$$I_{t/1} = I_{2/1} \cdot I_{3/2} \cdots I_{t/t-1} \quad I_{t/t-1} = \frac{y_t}{y_{t-1}} = \frac{I_{t/1}}{I_{(t-1)/1}}$$

7.3 Klouzavé průměry

7.3.1 Prostý klouzavý průměr (liché m)

Symetrické okno šířky $m = 2p + 1$: $\tilde{y}_t = \frac{1}{m} \sum_{j=-p}^p y_{t+j}$. Odstraňuje sezónnost délky $\leq m$.

Implementace v R

Funkce	Příklad použití	Vysvětlení
<code>filter(y, rep(1/m, m))</code>	<pre>m <- 5 # liche m -- prosty KP kp <- filter(y, rep(1/m,m), sides=2)</pre>	Prostý klouzavý průměr; <code>sides=2</code> = symetrické okno.

Implementace v Python

Funkce	Příklad použití	Vysvětlení
<code>np.convolve</code>	<pre>np.convolve(y, np.ones(m)/m, mode='valid')</pre>	Prostý klouzavý průměr (ztratí krajní hodnoty).

7.3.2 Centrovaný klouzavý průměr (sudé m)

Pro sudé m : centrovaný KP = průměr dvou po sobě jdoucích prostých KP délky m . Používá se pro čtvrtletní nebo měsíční data.

$$\bar{y}_t = \frac{y_{t-p} + 2y_{t-p+1} + \dots + 2y_t + \dots + 2y_{t+p-1} + y_{t+p}}{2m}, \quad m = 2p$$

Implementace v R

Funkce	Příklad použití	Vysvětlení
Centrovaný KP (sudé m)	<pre>m <- 4 # sude m -- centrovani KP kp_c <- filter(y, c(0.5, rep(1, m-1), 0.5)/m, sides=2)</pre>	Váhy <code>c(0.5, 1, ..., 1, 0.5)/m</code> odpovídají vzorci; <code>sides=2</code> = symetrické okno, výsledek je správně zarovnán na t .

Implementace v Python

Funkce	Příklad použití	Vysvětlení
Centrovaný KP	<pre># Pro m=4 (ctvrtletni data) m = 4 w = np.array([0.5] + [1]*(m-1) + [0.5]) / m kp_c = np.convolve(y, w[::-1], mode='valid')</pre>	Váhy <code>[0.5, 1, ..., 1, 0.5]/m</code> odpovídají vzorci; <code>mode='valid'</code> ořízne krajní hodnoty stejně jako <code>filter()</code> v R.

7.4 Trend a sezónnost – regresní přístup

7.4.1 Dummy proměnné a model

Model: $y_t = \beta_0 + \beta_1 t + \gamma_1 D_1 + \gamma_2 D_2 + \gamma_3 D_3 + \varepsilon_t$ (sezónnost délky 4; D_j jsou dummy proměnné; 4. čtvrtletí je referenční kategorie).

Implementace v R

Funkce	Příklad použití	Vysvětlení
Dummy proměnné	<pre>t <- 1:length(y) Q2 <- as.integer(t %% 4 == 2) Q3 <- as.integer(t %% 4 == 3) Q4 <- as.integer(t %% 4 == 0) # Q1 je referencni (t %% 4 == 1)</pre>	Vždy o jednu méně dummy než sezón – jinak hrozí multikolinearita.
lm() s trendem a sezónností	<pre>m <- lm(y ~ t + Q2 + Q3 + Q4) summary(m) fitted(m) # vyrovnané hodnoty</pre>	Odhaduje lineární trend a sezónní odchylky; $\hat{\gamma}_j$ = odchylka j -tého čtvrtletí od referenčního.

Implementace v Python

Funkce	Příklad použití	Vysvětlení
Dummy proměnné	<pre>import numpy as np t = np.arange(1, len(y)+1) Q2 = (t % 4 == 2).astype(int) Q3 = (t % 4 == 3).astype(int) Q4 = (t % 4 == 0).astype(int) # Q1 je referencni (t % 4 == 1)</pre>	Vždy o jednu méně dummy než sezón – jinak hrozí multikolinearita.
smf.ols() s trendem a sezónností	<pre>import pandas as pd import statsmodels.formula.api as smf df = pd.DataFrame({'y':y, 't':t, 'Q2':Q2, 'Q3':Q3, 'Q4':Q4}) m = smf.ols('y ~ t + Q2 + Q3 + Q4', data=df).fit() print(m.summary()) m.fittedvalues # vyrovnané hodnoty</pre>	Odhaduje lineární trend a sezónní odchylky; $\hat{\gamma}_j$ = odchylka j -tého čtvrtletí od referenčního.

7.4.2 Dekompozice časové řady

Rozklad časové řady na trend T_t , sezónní S_t , cyklickou C_t a reziduální složku ε_t :

$$y_t = T_t + S_t + C_t + \varepsilon_t \quad (\text{aditivní model}) \qquad y_t = T_t \cdot S_t \cdot C_t \cdot \varepsilon_t \quad (\text{multiplikativní model})$$

Rezidua a kvalita modelu (trend $\hat{y}_t$, MSE, Durbin-Watson D):

$$e_t = y_t - \hat{y}_t, \quad MSE = \frac{\sum_{t=1}^n e_t^2}{n}, \quad D = \frac{\sum_{t=2}^n (e_t - e_{t-1})^2}{\sum_{t=1}^n e_t^2}$$

Sezónní složky (regresní přístup, sezónnost délky 4; a_i jsou odhadnuté koeficienty dummy proměnných):

$$\bar{a} = \frac{a_1 + a_2 + a_3}{4}, \quad S_{i+4j} = a_i - \bar{a} \quad (i = 1, 2, 3), \quad S_{4+4j} = -\bar{a}$$

Implementace v Python

Funkce	Příklad použití	Vysvětlení
<code>seasonal_decompose</code>	<pre>from statsmodels.tsa.seasonal import seasonal_decompose decomp = seasonal_decompose(df['y'], model='additive', period=4) decomp.plot()</pre>	Automatický rozklad řady na komponenty. Vykreslí originální data, trend, sezónnost a rezidua.
Funkce	Příklad použití	Vysvětlení
<code>pd.get_dummies</code>	<pre>pd.get_dummies(df['quarter'], drop_first=True)</pre>	Automatické dummy proměnné z kategorického sloupce (pandas).

7.4.3 Predikce na budoucí období

Sestavíme `data.frame` s t a dummy proměnnými pro každé nové pozorování.

Implementace v R

Funkce	Příklad použití	Vysvětlení
<code>predict(m, newdata=...)</code>	<pre>t_n <- 21:24 nd <- data.frame(t = t_n, Q2 = as.integer(t_n %% 4 == 2), Q3 = as.integer(t_n %% 4 == 3), Q4 = as.integer(t_n %% 4 == 0)) predict(m, newdata = nd)</pre>	Predikce na budoucí kvartály.

Implementace v Python

Funkce	Příklad použití	Vysvětlení
<code>model.predict(X_new)</code>	<code>model.predict(X_new)</code>	Predikce na nová data (scikit-learn nebo statsmodels).

7.5 Nelineární trendy časových řad

7.5.1 Kvadratický (parabolický) trend

Model: $\hat{T}_t = b_0 + b_1 t + b_2 t^2$, kde $t = 1, 2, \dots, n$ je časová proměnná.

Znaménko b_2 určuje tvar paraboly: $b_2 > 0$ – konvexní (tvar U, minimum), $b_2 < 0$ – konkávní (tvar $\cap$, maximum).

Implementace v R

Funkce	Příklad použití	Vysvětlení
Kvadratický trend	<pre>t <- 1:length(y) m <- lm(y ~ t + I(t^2)) summary(m) # b0, b1, b2, R^2 fitted(m) # vyrovnané hodnoty</pre>	Funkce $I(t^2)$ izoluje mocninu od vzorcové syntaxe. Koeficienty v <code>Estimate:</code> (<code>Intercept</code>) = b_0 , $t = b_1$, $I(t^2) = b_2$.
Predikce	<pre># Predikce pro t = 11 (rok 2018) predict(m, newdata = data.frame(t = 11))</pre>	Dosadí $t = 11$ do odhadnutého modelu $\hat{T}_{11} = b_0 + b_1 \cdot 11 + b_2 \cdot 121$.

Implementace v Python

Funkce	Příklad použití	Vysvětlení
Kvadratický trend	<pre>import numpy as np import statsmodels.formula.api as smf t = np.arange(1, len(y) + 1) df['t'] = t df['t2'] = t**2 m = smf.ols('y ~ t + t2', data=df).fit() print(m.summary())</pre>	Kvadratický člen se přidá jako nový sloupec <code>t2</code> ; <code>summary()</code> vrátí b_0, b_1, b_2 a R^2 .

7.5.2 Exponenciální trend

Vhodný tehdy, kdy časová řada roste (nebo klesá) relativně konstantním tempem.

Model v logaritmické linearizaci:

$$y_t = b_0 \cdot b_1^t \implies \ln y_t = \ln b_0 + t \cdot \ln b_1 \implies y^* = \beta_0^* + \beta_1^* t$$

kde $y^* = \ln y_t$, $\beta_0^* = \ln b_0$, $\beta_1^* = \ln b_1$. Zpětná transformace: $b_0 = e^{\beta_0^*}$, $b_1 = e^{\beta_1^*}$. Tempo růstu: $(b_1 - 1) \times 100\%$ za jednu časovou jednotku.

Implementace v R

Funkce	Příklad použití	Vysvětlení
Exponenciální trend	<pre>t <- 1:length(y) m_exp <- lm(log(y) ~ t) summary(m_exp) # beta0*, beta1* # Zpetna transformace b0 <- exp(coef(m_exp)[1]) # b0 b1 <- exp(coef(m_exp)[2]) # b1</pre>	Regrese probíhá na $\ln y$; výsledné koeficienty je nutné zpětně transformovat pomocí $e^{(\cdot)}$.
Predikce (orig. škála)	<pre># Predikce pro t = 11 y_hat_log <- predict(m_exp, newdata = data.frame(t = 11)) exp(y_hat_log) # zpet na puvodni skalu</pre>	<code>predict</code> vrací hodnotu na logaritmické škále; <code>exp()</code> převede zpět na původní.

Implementace v Python

Funkce	Příklad použití	Vysvětlení
Exponenciální trend	<pre>import numpy as np import statsmodels.formula.api as smf df['log_y'] = np.log(df['y']) df['t'] = np.arange(1, len(df) + 1) m = smf.ols('log_y ~ t', data=df).fit() b0 = np.exp(m.params['Intercept ']) b1 = np.exp(m.params['t']) # Predikce y_pred = np.exp(m.predict(df_new))</pre>	Linearizace přes <code>log_y</code> ; zpětná transformace pomocí <code>np.exp()</code> .

8 Indexní analýza

Indexní analýza srovnává hodnoty ekonomických veličin (ceny, množství, výdaje) mezi dvěma obdobími: **základním** (index 0) a **běžným** (index 1).

Základní vztahy: $Q = p \cdot q$ (výdaj = cena $\times$ množství), $IQ = Ip \cdot Iq$

$$Ip = \frac{p_1}{p_0}, \quad \Delta p = p_1 - p_0, \quad Iq = \frac{q_1}{q_0}, \quad \Delta q = q_1 - q_0, \quad IQ = \frac{Q_1}{Q_0}, \quad \Delta Q = Q_1 - Q_0$$

8.1 Složené individuální indexy

Agregují individuální změny přes všechny druhy zboží.

$$I(\Sigma q) = \frac{\sum q_1}{\sum q_0} = \frac{\sum Iq \cdot q_0}{\sum q_0} = \frac{\sum q_1}{\sum \frac{q_1}{Iq}}, \quad \Delta(\Sigma q) = \sum q_1 - \sum q_0$$

$$I(\Sigma Q) = \frac{\sum Q_1}{\sum Q_0} = \frac{\sum p_1 q_1}{\sum p_0 q_0} = \frac{\sum IQ \cdot Q_0}{\sum Q_0} = \frac{\sum Q_1}{\sum \frac{Q_1}{IQ}}, \quad \Delta(\Sigma Q) = \sum Q_1 - \sum Q_0$$

$$I\bar{p} = \frac{\bar{p}_1}{\bar{p}_0} = \frac{\frac{\sum Q_1}{\sum q_1}}{\frac{\sum Q_0}{\sum q_0}} = \frac{\sum p_1 q_1}{\sum p_0 q_0}, \quad \Delta\bar{p} = \bar{p}_1 - \bar{p}_0 = \frac{\sum p_1 q_1}{\sum q_1} - \frac{\sum p_0 q_0}{\sum q_0}$$

kde $\bar{p}$ je vážený průměr cen (**weighted.mean**), kde váhy jsou množství q : v období 0 jsou váhy q_0 , v období 1 jsou váhy q_1 .

Implementace v R

Funkce	Příklad použití	Vysvětlení
Individuální indexy	<pre>Ip <- p1 / p0 Iq <- q1 / q0 IQ <- (p1 * q1) / (p0 * q0)</pre>	Individuální cenový, množství a výdajový index pro každé zboží.
Složené individ. indexy	<pre>I_sum_q <- sum(q1) / sum(q0) I_sum_Q <- sum(p1*q1) / sum(p0*q0) I_p_bar <- weighted.mean(p1, q1) / weighted.mean(p0, q0)</pre>	Složené individuální indexy součtů množství, výdajů a průměrné ceny.

Implementace v Python

Funkce	Příklad použití	Vysvětlení
Individuální indexy	<pre>Ip = p1 / p0 Iq = q1 / q0 IQ = (p1 * q1) / (p0 * q0)</pre>	Individuální cenový, množství a výdajový index pro každé zboží.
Složené individ. indexy	<pre>I_sum_q = q1.sum() / q0.sum() I_sum_Q = (p1*q1).sum() / (p0*q0).sum() I_p_bar = (np.average(p1, weights=q1) / np.average(p0, weights=q0))</pre>	Složené individuální indexy součtů množství, výdajů a průměrné ceny.

8.2 Souhrnné (Laspeyresovy a Paascheho) indexy

Souhrnné indexy fixují váhy buď v základním období (**Laspeyresovy**, superindex L), nebo v běžném období (**Paascheho**, superindex P).

Souhrnné cenové indexy:

$$I_p^{(L)} = \frac{\sum p_1 q_0}{\sum p_0 q_0} = \frac{\sum I_p \cdot p_0 q_0}{\sum p_0 q_0} = \frac{\sum I_p \cdot Q_0}{\sum Q_0} \quad I_p^{(P)} = \frac{\sum p_1 q_1}{\sum p_0 q_1} = \frac{\sum p_1 q_1}{\sum \frac{p_1 q_1}{I_p}} = \frac{\sum Q_1}{\sum \frac{Q_1}{I_p}}$$

Souhrnné množství indexy:

$$I_q^{(L)} = \frac{\sum p_0 q_1}{\sum p_0 q_0} = \frac{\sum I_q \cdot p_0 q_0}{\sum p_0 q_0} = \frac{\sum I_q \cdot Q_0}{\sum Q_0} \quad I_q^{(P)} = \frac{\sum p_1 q_1}{\sum p_1 q_0} = \frac{\sum p_1 q_1}{\sum \frac{p_1 q_1}{I_q}} = \frac{\sum Q_1}{\sum \frac{Q_1}{I_q}}$$

kde p_0, p_1 jsou ceny v základním a běžném období, q_0, q_1 jsou množství, $Q_0 = p_0 q_0$, $Q_1 = p_1 q_1$ jsou výdaje.

Implementace v R

Funkce	Příklad použití	Vysvětlení
Laspeyresovy indexy	<pre>IpL <- sum(p1 * q0) / sum(p0 * q0) IqL <- sum(p0 * q1) / sum(p0 * q0)</pre>	Laspeyresův cenový a množství index (váhy z období 0).
Paascheho indexy	<pre>IpP <- sum(p1 * q1) / sum(p0 * q1) IqP <- sum(p1 * q1) / sum(p1 * q0)</pre>	Paascheho cenový a množství index (váhy z období 1).

Implementace v Python

Funkce	Příklad použití	Vysvětlení
Laspeyresovy indexy	<pre>IpL = (p1 * q0).sum() / (p0 * q0).sum() IqL = (p0 * q1).sum() / (p0 * q0).sum()</pre>	Laspeyresův cenový a množstevní index (váhy z období 0).
Paascheho indexy	<pre>IpP = (p1 * q1).sum() / (p0 * q1).sum() IqP = (p1 * q1).sum() / (p1 * q0).sum()</pre>	Paascheho cenový a množstevní index (váhy z období 1).

8.3 Fisherův (ideální) index

Fisherův index je geometrický průměr Laspeyresova a Paascheho indexu. Platí $I_p^{(F)} \cdot I_q^{(F)} = I(\Sigma Q)$ (faktorový rozklad).

$$I_p^{(F)} = \sqrt{I_p^{(L)} \cdot I_p^{(P)}}, \quad I_q^{(F)} = \sqrt{I_q^{(L)} \cdot I_q^{(P)}}$$

Implementace v R

Funkce	Příklad použití	Vysvětlení
Fisherův index	<pre>IpF <- sqrt(IpL * IpP) IqF <- sqrt(IqL * IqP)</pre>	Geometrický průměr Laspeyresova a Paascheho indexu.

Implementace v Python

Funkce	Příklad použití	Vysvětlení
Fisherův index	<pre>IpF = np.sqrt(IpL * IpP) IqF = np.sqrt(IqL * IqP)</pre>	Geometrický průměr Laspeyresova a Paascheho indexu.

9 Příloha: Utility funkce v Pythonu

9.1 Užitečné knihovny a základy

Funkce	Příklad použití	Vysvětlení
<code>import numpy as np</code>	<code>np.array([...])</code>	Matice, vektory a rychlé matematické operace.
<code>import pandas as pd</code>	<code>pd.DataFrame(matice)</code>	Analýza tabulek a práce s CSV.
<code>import matplotlib.pyplot as plt</code>	<code>plt.hist(data)</code>	Tvorba grafů a vizualizace dat.
<code>math.comb(n, k)</code>	<code>math.comb(10, 3)</code>	Kombinační číslo (výběr k z n).
List Comprehension	<code>[x**2 for x in range(5)]</code>	Dynamické generování pole pomocí cyklu.

9.2 Práce s poli (Standardní Python)

Funkce	Příklad použití	Vysvětlení
List Comprehension	<code>[x**2 for x in range(5)]</code>	Dynamické generování pole pomocí cyklu.
<code>x[start:stop:krok]</code>	<code>pole[1:10:2]</code>	Výřez (stop je exkluzivní). <code>[::-1]</code> pro otočení.
<code>x.copy()</code> vs <code>deepcopy</code>	<code>a = b.copy()</code>	<code>copy</code> je plytká, <code>deepcopy</code> kopíruje i vnořená pole.
<code>x.append(v)</code>	<code>data.append(10)</code>	Přidání jednoho prvku na konec pole.
<code>x.extend(a)</code>	<code>data.extend([1,2,3])</code>	Přidání více prvků na konec pole.

9.3 Základní Python (Built-in & Math)

Funkce	Příklad použití	Vysvětlení
<code>help(funkce)</code>	<code>help(print)</code>	Dokumentace pro zadanou funkci.
<code>range(start, stop)</code>	<code>list(range(0, 5))</code>	Vygeneruje sekvenci [0, 1, 2, 3, 4].
<code>math.log(x, b)</code>	<code>math.log(8, 2)</code>	Logaritmus čísla x o základu b .
<code>math.sqrt(x)</code>	<code>math.sqrt(16)</code>	Druhá odmocnina čísla.
<code>math.ceil / floor</code>	<code>math.ceil(2.1)</code>	Zaokrouhlení nahoru (3) / dolů (2).
<code>math.trunc(x)</code>	<code>math.trunc(2.9)</code>	Odříznutí desetinné části (2).
<code>math.comb(n, k)</code>	<code>math.comb(10, 3)</code>	Kombinační číslo (počet kombinací k prvků z n).
<code>def function(...):</code>	<code>def funkce(x, y, z):</code>	Definice vlastní funkce s parametry.

9.4 Statistika a Matice (Knihovna NumPy a jiné)

Funkce	Příklad použití	Vysvětlení
<code>np.sum(x)</code>	<code>np.sum(data, axis=0)</code>	Součet všech prvků v poli (nebo podél zvolené osy).
<code>np.mean(x)</code>	<code>np.mean(data)</code>	Průměr všech prvků v poli.
<code>np.prod(x)</code>	<code>np.prod(data)</code>	Součin všech prvků v poli.
<code>np.sqrt(x)</code>	<code>np.sqrt(data)</code>	Druhá odmocnina z každého prvku v poli.
<code>np.random.rand(n)</code>	<code>np.random.rand(100)</code>	Generování pole n náhodných čísel z rovnoměrného rozdělení na $[0, 1)$.
<code>np.random.uniform(a,b,n)</code>	<code>np.random.uniform(10, 50, 100)</code>	Generuje n náhodných desetinných čísel v rozsahu $[a, b)$.
<code>np.quantile(x, q)</code>	<code>np.quantile(x, 0.75, method='midpoint')</code>	Výpočet kvantilu ($q \in [0, 1]$).
Modus	<code>statistics.multimode(data)</code>	Vrací modus z pole (i více prvků)
<code>np.log10(x)</code>	<code>np.log10(data)</code>	Dekadický logaritmus (základ 10). Často se používá pro normalizaci dat.
<code>np.linalg.norm(x, p)</code>	<code>np.linalg.norm(x, ord=2)</code>	Výpočet L^1, L^2 nebo L^∞ (<code>np.inf</code>) normy.
<code>np.insert / np.delete</code>	<code>np.insert(A, 1, [...], axis=0)</code>	Vloží/Smaže řádek (<code>axis=0</code>) nebo sloupec (<code>axis=1</code>).

Funkce	Příklad použití	Vysvětlení
<code>stats.gmean(x)</code>	<code>stats.gmean(x)</code>	Geometrický průměr (knihovna Scipy).
<code>stats.hmean(x)</code>	<code>stats.hmean(x, weights=[...])</code>	Harmonický průměr (knihovna Scipy).
<code>np.var(x, ddof=0)</code>	<code>np.var(data, ddof=0)</code>	Klasický rozptyl s_x^2 (děleno n).
<code>np.var(x, ddof=1)</code>	<code>np.var(data, ddof=1)</code>	Výběrový rozptyl S^2 (děleno $n - 1$).
<code>np.std(x, ddof=0)</code>	<code>np.std(data, ddof=0)</code>	Klasická směrodatná odchylka s_x .
<code>np.std(x, ddof=1)</code>	<code>np.std(data, ddof=1)</code>	Výběrová směrodatná odchylka S .
<code>v_x = std / mean</code>	<code>np.std(x, ddof=1) / np.mean(x)</code>	Variační koeficient v_x (relativní variabilita).
<code>np.where(cond, x, y)</code>	<code>np.where(df['am'] == 0, 'aut', 'man')</code>	Podmíněný sloupec: Pokud platí <code>cond</code> , vlož <code>x</code> , jinak <code>y</code> (jako IF v Excelu).

9.5 Pandas: Analýza dat

Funkce	Příklad použití	Vysvětlení
<code>pd.to_datetime()</code>	<pre>df['datum'] = pd.to_datetime(df['datum']) df.set_index('datum', inplace=True)</pre>	Převod textového sloupce na datumové hodnoty a jeho následné nastavení jako indexu tabulky (klíčové pro časové řady).
<code>pd.date_range()</code>	<pre>pd.date_range(start='2020-01-01', periods=10, freq='Q')</pre>	Vygeneruje časovou osu, zde například se čtvrtletní ('Q') frekvencí.
<code>pd.read_csv(file)</code>	<pre>pd.read_csv('data.csv')</pre>	Načte data z CSV souboru a vytvoří DataFrame.
<code>df.describe()</code>	<pre>df.describe()</pre>	Souhrnné statistiky (průměr, std, kvartily, atd.).
<code>df[columns]</code>	<pre>df[["A", "B", "C"]]</pre>	Výběr určitých sloupců.
Filtrování	<pre>df[df['Věk'] >= 18]</pre>	Vybere pouze řádky splňující danou podmínku.
<code>df['A'].isin([list])</code>	<pre>df[df['mesto'].isin(['Praha', 'Brno'])]</pre>	Vybere řádky, kde hodnota sloupce odpovídá prvkům v seznamu.
<code>df.loc</code> <code>df.iloc</code>	vs <pre>df.loc[0, 'Sloupec']</pre>	Výběr dat podle názvů (loc) vs. pozice (iloc).

Funkce	Příklad použití	Vysvětlení
<code>df.isna().sum()</code>	<code>df.isna().sum()</code>	Počet chybějících (null) hodnot v tabulce.
<code>df.value_counts()</code>	<code>df['A'].value_counts()</code>	Tabulka četností pro kategorická data.
<code>df.groupby('A')</code>	<code>df.groupby('A').mean()</code>	Výpočet statistik podle kategorií ve sloupci 'A'.
<code>df.cumsum()</code>	<code>df['cetnost'].cumsum()</code>	Kumulativní součet: Pro výpočet kumulativních absolutních/relativních četností.
<code>pd.crosstab()</code>	<code>pd.crosstab(df['A'], df['B'])</code>	Kontingenční tabulka četností dvou kategorií.
<code>pd.pivot_table()</code>	<code>df.pivot_table(values='C', \dots)</code>	Univerzální pivotní tabulka.
<code>df['A'].between(a, b)</code>	<code>df[df['mzda'].between(30000, 50000)]</code>	Filtrování hodnot v uzavřeném intervalu [a, b].

9.6 Matplotlib a Seaborn: Vizualizace

Funkce	Příklad použití	Vysvětlení
<code>df.boxplot()</code>	<pre>df.boxplot(column='Y', by='X')</pre>	Boxplot (Pandas): Rozdělení dat do kvartilů dle skupiny.
<code>plt.hist(x)</code>	<pre>plt.hist(data, bins=15)</pre>	Histogram: Rozdělení četností. <code>bins</code> určuje intervaly.
<code>plt.scatter(x, y)</code>	<pre>plt.scatter(x, y, alpha=0.5)</pre>	Bodový graf: Závislost dvou proměnných.
<code>plt.title(s)</code>	<pre>plt.title("Název", fontsize=16)</pre>	Nastaví hlavní nadpis grafu.
<code>plt.xlabel / ylabel</code>	<pre>plt.xlabel("Čas [s]")</pre>	Nastaví popisky os <i>x</i> a <i>y</i> .
<code>plt.legend()</code>	<pre>plt.legend(loc='upper right')</pre>	Zobrazí legendu (nutný parametr <code>label</code> u dat).
<code>plt.savefig()</code>	<pre>plt.savefig("graf.pdf")</pre>	Uloží graf na disk.
<code>sns.regplot()</code>	<pre>import seaborn as sns sns.regplot(data=df, x='TV', y='Sales')</pre>	Vykreslí bodový graf (<code>scatter</code>) a rovnou jím proloží regresní přímku (včetně intervalu spolehlivosti).

10 Příloha: Utility funkce v R

10.1 Užitečné balíčky

Funkce	Příklad použití	Vysvětlení
<code>library(stats)</code>	<code>library(stats)</code>	Základní statistické funkce (součást R) — <code>t.test</code> , <code>chisq.test</code> , <code>aov</code> , <code>lm</code> , ...
<code>library(ggplot2)</code>	<code>library(ggplot2)</code>	Vizualizace dat (grafy).
<code>library(dplyr)</code>	<code>library(dplyr)</code>	Manipulace s daty — <code>filter</code> , <code>select</code> , <code>mutate</code> , <code>group_by</code> , <code>summarise</code> .
<code>library(readr)</code>	<code>library(readr)</code>	Načítání CSV a textových souborů.

10.2 Vektory a datové typy

Datové typy a konverze

Funkce	Příklad použití	Vysvětlení
<code>class(x)</code>	<code>class(3.14) # "numeric"</code> <code>class(TRUE) # "logical"</code> <code>class("ahoj") # "character"</code>	Vrátí datový typ objektu.
<code>is.numeric /</code> <code>is.character</code> ...	<code>is.numeric(3.14) # TRUE</code> <code>is.integer(3L) # TRUE</code> <code>is.logical(FALSE) # TRUE</code>	Ověří datový typ; vrací TRUE/FALSE.
<code>as.numeric(x)</code>	<code>as.numeric("3.14") # 3.14</code> <code>as.integer(3.7) # 3</code> <code>as.character(42) # "42"</code> <code>as.logical(0) # FALSE</code>	Konverze na jiný typ.
<code>is.na(x)</code>	<code>is.na(NA) # TRUE</code> <code>is.na(NaN) # TRUE</code> <code>is.na(Inf) # FALSE</code> <code>colSums(is.na(df)) # NA po sl.</code>	Detekce chybějící hodnoty (NA/NaN).

Vektory

Funkce	Příklad použití	Vysvětlení
<code>c(...)</code>	<pre>c(1, 2, 3) c("a", "b") c(x, y) # spojení vektorů</pre>	Vytvoří vektor nebo spojí vektory.
<code>seq(from, to, by)</code>	<pre>1:10 seq(2, 10, by = 2) # 2,4,6,8,10</pre>	Vytvoří sekvenci čísel.
<code>x[i]</code>	<pre>x[1] # první prvek x[1:3] # prvky 1-3 x[-2] # vše kromě 2. x[x > 5] # podmíněný výběr</pre>	Indexování vektoru.
<code>length(x) / rev(x)</code>	<pre>length(c(1,2,3)) # 3 rev(c(1,2,3)) # 3 2 1</pre>	Délka vektoru a obrácení pořadí.
<code>names(x)</code>	<pre>names(x) <- c("a","b","c") x["a"] # výběr dle jména</pre>	Pojmenování prvků vektoru.

10.3 Práce s daty v R

Data frames

Funkce	Příklad použití	Vysvětlení
<code>str(df)</code> / <code>summary(df)</code>	<pre>str(mtcars) summary(mtcars) dim(mtcars) # c(32, 11)</pre>	Struktura, přehled a rozměry tabulky.
<code>head(df, n)</code> / <code>tail(df, n)</code>	<pre>head(mtcars) # prvních 6 head(mtcars, 10) # prvních 10 tail(mtcars) # posledních 6</pre>	Náhled na prvních / posledních n řádků.
<code>df[r, c]</code> / <code>df\$col</code>	<pre>df[2, 3] # jeden prvek df[1,] # 1. řádek df[, 2] # 2. sloupec df\$mpg # sloupec mpg df[df\$mpg > 20,] # podmíněný výběr</pre>	Indexování data.frameu.
<code>colSums()</code> / <code>colMeans()</code>	<pre>colSums(df) colMeans(df, na.rm = TRUE) apply(df, 2, mean, na.rm = TRUE)</pre>	Součet / průměr každého sloupce.
<code>cbind()</code> / <code>rbind()</code>	<pre>cbind(df, novy = c(...)) # sloupec rbind(df, c(...)) # řádek df[-1,] # odebrání 1. řádku df[, -2] # odebrání 2. sloupce</pre>	Přidání nebo odebrání řádků / sloupců.

Import a export dat

Funkce	Příklad použití	Vysvětlení
<code>read.table()</code>	<pre>read.table("data.csv", sep = ";", header = TRUE)</pre>	Načte textový soubor do data.frameu.
<code>read.csv()</code>	<pre>read.csv("data.csv") read.csv("data.csv", sep = ",")</pre>	Zkratka pro CSV soubory.
<code>write.table()</code>	<pre>write.table(mtcars, file = "out.csv", sep = ";", row.names = FALSE)</pre>	Uloží data.frame do textového souboru.
<code>getwd()</code> / <code>setwd()</code>	<pre>getwd() setwd("C:/Users/.../data")</pre>	Zjistí nebo nastaví pracovní složku.

Četnosti a diskretizace

Funkce	Příklad použití	Vysvětlení
<code>table(x)</code>	<pre>ni <- table(data\$kraj) pi <- prop.table(ni)</pre>	Absolutní a relativní četnosti.
<code>cumsum(x)</code>	<pre>Ni <- cumsum(ni) Pi <- cumsum(pi)</pre>	Kumulativní (absolutní / relativní) četnosti.
<code>cut(x, breaks)</code>	<pre>cut(x, breaks = seq(0, 20, by = 2), include.lowest = TRUE)</pre>	Diskretizuje spojitou proměnnou do intervalů.

Balíček dplyr (`|>` = roura: výsledek vlevo jde jako 1. argument vpravo)

Funkce	Příklad použití	Vysvětlení
<code>filter()</code>	<pre>mtcars > filter(mpg > 20) mtcars > filter(mpg > 20, cyl == 6)</pre>	Výběr řádků splňujících podmínku.
<code>select()</code>	<pre>mtcars > select(mpg, cyl, hp)</pre>	Výběr (podmnožiny) sloupců.
<code>mutate()</code>	<pre>mtcars > mutate(pwr_wt = hp / wt)</pre>	Přidá nový sloupec odvozený od existujících.
<code>summarise()</code>	<pre>mtcars > summarise(prumer = mean(mpg), odch = sd(mpg))</pre>	Souhrnné statistiky (redukuje tabulku na 1 řádek).
<code>group_by()</code>	<pre>mtcars > group_by(cyl) > summarise(n = n(), prumer = mean(mpg))</pre>	Seskupení řádků; summarise pak počítá za každou skupinu.
<code>count()</code>	<pre>mtcars > count(cyl) data > count(kraj) > arrange(desc(n))</pre>	Absolutní četnosti pro každou hodnotu.
<code>arrange()</code>	<pre>mtcars > arrange(mpg) mtcars > arrange(desc(mpg))</pre>	Seřadí řádky vzestupně / sestupně.
<code>pull()</code>	<pre>mtcars > filter(cyl==6) > pull(mpg)</pre>	Extrahuje sloupec jako prostý vektor.